DOCUMENTATION SECTION

Technical Documentation

Full implementation reference — core engine architecture, PAM contract processor, state machine, financial conventions, schedule generation, GPU acceleration, demos, and testing.

- Core Engine — PAM Contract, State Machine, Conventions
- Schedule Generation & Cycle Parsing
- Risk Factor Model & Market Data
- GPU Acceleration — Data Structures, Kernels, Executors
- Demos & Sample Applications
- Testing — 42 PAM Tests, GPU Validation
- Technical Reference — Types, Enums, Interfaces

Table of Contents

Technical Documentation

- Core Engine Overview
- GPU Architecture
- Testing Overview
- PAM Contract Implementation
- Scenario Demo — CPU vs GPU Comparison
- GPU Data Structures
- Running Tests
- State Machine
- GPU Kernels
- GPU Executors
- Schedule Generation
- Risk Factor Model
- Technical Reference
- Financial Conventions
- CLI Demo Tool — Monte Carlo Portfolio Valuation

Technical

Technical Documentation

Technical Documentation

Overview

This section covers the implementation details of the ACTUS execution engine. It is organized into four areas: the core engine (CPU implementation), GPU acceleration, demonstration tools, and testing.

Each area starts with a conceptual overview and progressively adds implementation detail. Developers who need to understand the codebase, contribute to it, or integrate it into other systems should start here.

Architecture at a Glance

The system is organized into four repositories, each with a specific responsibility:

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    subgraph "Actus-Insurance.Core"
        CORE[Core Library<br/>Models, Types, Conventions]
        CPU[CPU Engine<br/>PAM Contract Implementation]
        TESTS_CPU[CPU Test Suite<br/>42 ACTUS PAM Tests]
    end

    subgraph "Actus-Insurance.GPU"
        GPU_LIB[GPU Library<br/>Kernels, Executors, Adapters]
        LIFE[Life Insurance GPU<br/>Markov Kernel, DSL, Tables]
        RUNNER[Portfolio Runner<br/>High-Level API]
        BENCH[Benchmarks<br/>CPU vs GPU Comparison]
        TESTS_GPU[GPU Test Suite<br/>Correctness Validation]
    end
```

```
    subgraph "Actus-Insurance.DemoAndSample"
        CLI[CLI Demo Tool<br/>Monte Carlo, Reporting]
        SAMPLE[Scenario Demo<br/>CPU/GPU Comparison]
    end

    subgraph "Actus-Insurance.Documentation"
        DOCS[Next.js Site<br/>Searchable Documentation]
    end

    CORE --> CPU
    CORE --> GPU_LIB
    CPU --> TESTS_CPU
    CPU --> GPU_LIB
    GPU_LIB --> LIFE
    GPU_LIB --> RUNNER
    GPU_LIB --> BENCH
    GPU_LIB --> TESTS_GPU
    RUNNER --> CLI
    GPU_LIB --> SAMPLE
```

Technology Stack

Component	Technology	Version
Runtime	.NET	9.0
Language	C#	13
GPU Framework	ILGPU	1.5.3
Test Framework	NUnit	Latest
Benchmark Framework	BenchmarkDotNet	Latest
Documentation Site	Next.js	16.0
Documentation Format	Markdown + Mermaid	—

Section Contents

Core Engine

The heart of the system: contract term models, event generation, state transitions, payoff computation, financial conventions, schedule generation, and market data handling.

- Core Engine Overview — architecture, design principles, and data flow

- PAM Contract — full PAM implementation: terms, schedule generation, event evaluation, payoff formulas
- State Machine — event processing, state transitions, interest accrual, rate reset algorithm
- Conventions — day count, business day, end-of-month, and calendar conventions
- Schedule Generation — cycle strings, periodic date sequences, stub handling
- Risk Factor Model — market data storage, LOCF lookup, usage patterns
- Technical Reference — complete listing of all types, enumerations, interfaces, and structs

GPU Acceleration

How the CPU engine was adapted for GPU execution, including data structures, kernels, executors, and memory management.

- GPU Architecture — overview of the GPU execution pipeline
- Data Structures — blittable structs and data packing
- Kernels — GPU kernel implementations
- Executors — device management and buffer pooling

Demos & Samples

Demonstration tools and sample applications that show how to use the engine.

- CLI Tool — Monte Carlo portfolio valuation CLI
- Scenario Demo — CPU vs GPU comparison tool

Testing

Test strategy, test data, and how to run the test suites.

- Testing Overview — test architecture and validation approach
- Running Tests — how to execute the test suites

Core Engine Overview

Core Engine Overview

Overview

The core engine is the heart of the ACTUS implementation. It takes a set of contract terms — the legal and financial parameters that define a financial instrument — and produces a complete, time-ordered list of evaluated events. Each event carries a date, a type, a cash flow amount (payoff), and a snapshot of the contract's state after the event occurred. Every other component in the system (GPU acceleration, Monte Carlo simulation, insurance extensions) builds on top of this deterministic evaluation pipeline.

The engine is split across two .NET projects that separate abstraction from implementation:

- **ActusInsurance.Core** — shared abstractions: interfaces, domain models, type enumerations, financial conventions (day count, business day, end-of-month, calendars), time utilities (schedule generation, cycle parsing), and the risk factor model. This project has zero external dependencies — it is pure C#.
- **ActusInsurance.Core.CPU** — contract-type implementations that use the Core abstractions. Currently contains the full PAM (Principal at Maturity) implementation: schedule generation and sequential event evaluation.

A third project, **ActusInsurance.Tests.CPU**, provides 42 reference test cases derived from the official ACTUS test suite in JSON format, validating every event type, convention, and edge case against known-correct results.

Design Principles

Three principles govern the engine's design:

Deterministic execution. Given the same contract terms and the same market data, the engine always produces exactly the same output. This is not optional — financial contracts must be auditable and independently verifiable. The engine achieves determinism through strict event ordering (time first, then priority), pure functions for payoff computation, and no dependence on floating-point operation order.

Separation of terms and logic. Contract terms are plain data objects with no behavior. All computation lives in the contract processor (e.g., `PrincipalAtMaturity`), which reads

terms but never modifies them. This separation makes it straightforward to serialize terms, send them to a GPU, or store them in a database.

Convention-driven computation. Financial contracts differ not just in their structure but in the conventions they use for dates, interest accrual, and calendar adjustments. Rather than hard-coding these rules, the engine parameterizes them: each contract specifies which day count convention, business day convention, end-of-month convention, and calendar to use. The engine dispatches to the correct implementation at runtime.

The Three-Stage Pipeline

Every contract evaluation follows the same three-stage pipeline:

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph LR
    Terms["Contract Terms<br/>(PamContractTerms)"] --> Schedule["Stage 1: Schedu
    Schedule --> Events["Raw Events<br/>(typed, timed, unsorted)"]
    Events --> Apply["Stage 2: Apply<br/>(sort, initialize, evaluate)"]
    Market["Risk Factor Model<br/>(market rates)"] --> Apply
    Apply --> Result["Stage 3: Result<br/>(evaluated events with payoffs)"]
```

Stage 1: Schedule Generation

The `Schedule()` method reads the contract terms and generates every event that will occur during the contract's lifetime. It creates mandatory events (Initial Exchange Date, Maturity Date) and then generates periodic events from cycle definitions (interest payments, rate resets, fee payments, scaling updates). Each event is stamped with two times: the original schedule time (the contractually specified date) and the shifted time (the date adjusted for business day conventions). Events are not yet sorted or evaluated — they are raw placeholders.

Stage 2: Event Application

The `Apply()` method takes the raw events, sorts them by time and priority, initializes the contract state from the terms, and then processes each event in strict sequence. For each event, the engine:

1. Accrues interest from the previous event's date to the current event's date
2. Computes the event's payoff using its type-specific formula
3. Updates the contract state using the event's type-specific transition logic
4. Records a snapshot of the payoff and the updated state into the event

The risk factor model is consulted whenever an event needs external market data (rate resets, scaling index lookups).

Stage 3: Result

The output is the same list of events, now populated with computed payoffs and state snapshots. Each event in the result contains: the event date, event type, payoff amount, and the full contract state (notional principal, nominal interest rate, accrued interest, fee accrued) as it stood immediately after the event was processed.

Key Abstractions

IContractScheduler

The generic interface that all contract types implement:

```
IContractScheduler<TTerms>
├── Schedule(DateTime to, TTerms terms) → List<ContractEvent>
└── Apply(List<ContractEvent> events, TTerms terms, RiskFactorModel) →
List<ContractEvent>
```

`TTerms` is the contract-specific terms type. For PAM, it is `PamContractTerms`. Other ACTUS contract types (LAM, NAM, ANN) would implement this interface with their own terms type, reusing the shared conventions and event infrastructure.

PamContractTerms

The domain model that carries all parameters of a PAM contract: over 40 fields organized into identity and dates, principal and interest, payment cycles, rate reset parameters, fee settings, scaling configuration, and convention selections. Terms are plain data — no behavior, no side effects. They can be deserialized from JSON using a `FromDictionary()` factory method that maps the official ACTUS test format directly.

ContractEvent

A sealed class representing a single event in a contract's life. Each event knows its type, its scheduled and adjusted times, and contains the logic to compute its own payoff and update the contract state via the `Evaluate()` method. Events implement `IComparable<ContractEvent>`, sorting first by adjusted time and then by event-type priority to ensure deterministic ordering.

StateSpace

A value struct that captures the complete state of a contract at a point in time: notional principal, nominal interest rate, accrued interest, fee accrued, notional and interest scaling multipliers, status date, and contract performance. Using a struct ensures copy-by-value semantics, which matters for thread safety when the same contract is evaluated under multiple scenarios in parallel.

RiskFactorModel

A lookup service for external market data. Stores rates as either constant values (a single rate for all times) or time series (rates keyed by date). When a time series lookup does not find an exact date match, it uses Last Observation Carried Forward (LOCF) — returning the most recent rate on or before the requested date.

Project Structure

```
ActusInsurance.Core/
├── Contracts/
│   └── IContractScheduler.cs          — Generic contract scheduler
interface
├── Conventions/
│   ├── BusinessDay/                    — BusinessDayAdjuster + 9
convention implementations
│   │   ├── BusinessDayAdjuster.cs      — Main coordinator
(ShiftEventTime, ShiftCalcTime)
│   │   ├── IBusinessDayConvention.cs   — Shift strategy interface
│   │   ├── IShiftCalcConvention.cs     — Calc/Shift strategy interface
│   │   ├── Same.cs                     — NOS: no adjustment
│   │   ├── Following.cs               — Move forward to next business
day
│   │   ├── ModifiedFollowing.cs        — Forward, but stay in same month
│   │   ├── Preceeding.cs              — Move backward to previous
business day
│   │   ├── ModifiedPreceeding.cs       — Backward, but stay in same
month
│   │   ├── ShiftCalc.cs               — Calculations use shifted dates
│   │   └── CalcShift.cs               — Calculations use original dates
│   ├── ContractRoles/
│   │   └── ContractRoleConvention.cs   — RoleSign computation (+1 or -1)
│   ├── DayCount/
│   │   ├── DayCountCalculator.cs       — Factory: string → convention
instance
│   │   ├── IDayCountConventionProvider.cs — Interface: DayCount +
DayCountFraction
```

```
│   │   ├── ActualThreeSixtyFiveFixed.cs — A/365: actual days ÷ 365
│   │   ├── ActualThreeSixty.cs         — A/360: actual days ÷ 360
│   │   ├── ActualActualISDA.cs         — A/A ISDA: leap-year-aware
│   │   ├── ThirtyEThreeSixty.cs        — 30E/360: day-31 → 30
│   │   ├── ThirtyEThreeSixtyISDA.cs    — 30E/360 ISDA: maturity-aware
variant
│   │   ├── BusinessTwoFiftyTwo.cs      — B/252: business days ÷ 252
│   │   ├── ActualThreeThirtySix.cs     — A/336: actual days ÷ 336
│   │   └── TwentyEightThreeThirtySix.cs — 28/336: day-28 convention
│   └── EndOfMonth/
│       ├── EndOfMonthAdjuster.cs       — Main coordinator (EOM vs SD)
│       ├── IEndOfMonthConvention.cs    — Shift interface
│       ├── EndOfMonth.cs               — Snap to last day of month
│       └── SameDay.cs                  — No adjustment
├── Events/
│   └── ContractEvent.cs                — Event with Evaluate,
ComputePayoff, UpdateState
├── Externals/
│   └── RiskFactorModel.cs              — Market data: constant rates +
time series + LOCF
├── Models/
│   ├── IContractTerms.cs               — Base interface (ContractID,
ContractType)
│   └── PamContractTerms.cs             — 40+ fields, FromDictionary()
factory
├── States/
│   └── StateSpace.cs                   — Value struct: 8 state variables
├── Time/
│   ├── ScheduleFactory.cs              — Periodic date sequence
generation
│   ├── CycleAdjuster.cs                — Period vs weekday cycle
dispatch
│   ├── PeriodCycleAdjuster.cs          — Period-based cycle arithmetic
│   ├── WeekdayCycleAdjuster.cs         — Weekday-based cycle arithmetic
│   ├── TimeAdjuster.cs                 — Hour rounding utility
│   └── Calendar/
│       ├── BusinessDayCalendarProvider.cs — Base: weekday check
│       ├── MondayToFridayCalendar.cs     — MF: Mon–Fri only
│       ├── MondayToFridayWithHolidaysCalendar.cs — MFH: Mon–Fri minus
holidays
│       └── NoHolidaysCalendar.cs         — NC: all days are business
days
├── Types/
│   └── Enums.cs                        — EventType, ContractRole,
DayCountConvention,
│                                          BusinessDayConventionEnum,
Calendar,
```

```
│                                          EndOfMonthConventionEnum,
ContractTypeEnum,
│                                          ContractPerformance, FeeBasis,
CyclePoints
└── Util/
    ├── CycleUtils.cs                  — Cycle string parsing (period,
stub, weekday)
    ├── Constants.cs                   — MAX_LIFETIME (50 years), stub
characters
    ├── StringUtils.cs                 — Convention string constants
    ├── CommonUtils.cs                 — Null-check helper
    └── AttributeConversionException.cs — Parsing error type

ActusInsurance.Core.CPU/
└── Contracts/
    └── PAM.cs                         — PrincipalAtMaturity: Schedule +
Apply (346 lines)

ActusInsurance.Tests.CPU/
├── PamTests.cs                        — 42 test cases from official
ACTUS suite
└── Resources/
    └── actus-tests-pam.json           — Test data: terms, market data,
expected results
```

How It All Connects

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    subgraph "ActusInsurance.Core (shared)"
        TERMS["PamContractTerms<br/>(40+ fields)"]
        CONV["Conventions<br/>(day count, business day,<br/>EOM, calendar)"]
        SCHED["ScheduleFactory<br/>(periodic date generation)"]
        EVENT["ContractEvent<br/>(Evaluate, ComputePayoff,<br/>UpdateState)"]
        STATE["StateSpace<br/>(8 state variables)"]
        RISK["RiskFactorModel<br/>(market data lookup)"]
    end

    subgraph "ActusInsurance.Core.CPU"
        PAM["PrincipalAtMaturity<br/>(Schedule + Apply)"]
    end

    subgraph "ActusInsurance.Tests.CPU"
        TESTS["42 PAM Test Cases<br/>(official ACTUS suite)"]
    end
```

```
    PAM --> TERMS
    PAM --> CONV
    PAM --> SCHED
    PAM --> EVENT
    PAM --> STATE
    PAM --> RISK
    TESTS --> PAM
```

Continue Reading

- PAM Contract — full walkthrough of the PAM implementation: terms, schedule generation, event evaluation, and all payoff formulas
- State Machine — how events modify contract state, with detailed transition logic for each event type
- Conventions — day count, business day, end-of-month, and calendar convention implementations
- Schedule Generation — how periodic date sequences are built from cycle strings
- Risk Factor Model — market data storage, lookup strategies, and the LOCF algorithm
- Technical Reference — complete listing of all types, enumerations, interfaces, and structs

GPU Architecture

GPU Architecture

Overview

The GPU execution pipeline transforms CPU domain objects into flat GPU-compatible structures, runs massively parallel kernels, and returns results. The architecture is organised into three layers: the adapter (data transformation), the kernel (GPU computation), and the executor (orchestration).

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph LR
    subgraph "CPU Domain"
        TERMS["PamContractTerms[]<br/>Rich domain objects"]
        MARKET["RiskFactorModel<br/>Market data"]
    end

    subgraph "Adapter Layer"
        ADAPT["PamV3Adapter<br/>Schedule + Flatten"]
    end

    subgraph "GPU Memory"
        CGPU["PamContractGpu[]"]
        EGPU["PamEventGpu[]"]
        RGPU["PamEventResultGpu[]"]
        MGPU["GpuMarketRate[]"]
    end

    subgraph "GPU Kernel"
        KERN["PamGpuKernel<br/>Per-thread evaluation"]
    end

    TERMS --> ADAPT
    MARKET --> ADAPT
    ADAPT --> CGPU
    ADAPT --> EGPU
    ADAPT --> MGPU
    CGPU --> KERN
    EGPU --> KERN
    MGPU --> KERN
    KERN --> RGPU
```

The Three Layers

Adapter (PamV3Adapter)

The adapter sits between the CPU domain model and the GPU. It performs two jobs: it generates the event schedule for each contract (using the same schedule logic as the CPU engine), and it flattens the results into GPU-compatible flat arrays.

The key transformation is the **event offset strategy**: all contracts' events are packed into a single contiguous array. Each contract record stores an offset and count that point to its events within this shared array. This avoids per-contract memory allocation on the GPU.

CPU Domain Object	GPU Representation
PamContractTerms (rich, with strings, dates, optional fields)	PamContractGpu (fixed-size struct, all numeric, offsets into event array)
List of ContractEvent per contract	Single flat PamEventGpu[] with offset/count per contract
RiskFactorModel (dictionary lookups)	GpuMarketRate[] flat array with index references
DateTime	long (ticks)
enum labels ("Annual", "Fixed")	int codes

Kernel (PamGpuKernel)

The kernel is a static method that runs on the GPU. Each thread processes one contract by iterating through its events, computing payoffs, and updating state. The kernel implements the same event logic as the CPU engine: IED, IP, IPCI, RR, RRF, FP, SC, MD, PRD, TD, AD, CD.

For Monte Carlo, the PayoffCubeKernel extends this to 2D indexing (contract × scenario), accumulating payoffs into a flat output cube.

Executor (PamGpuExecutor)

The executor manages the ILGPU context, accelerator selection (CUDA → OpenCL → CPU fallback), buffer allocation, H2D/D2H transfers, and kernel dispatch. It supports two modes:

- **EvaluateBatch** — single-call pipeline: allocate, upload, execute, download, return
- **UploadAndExecute + DownloadResults** — split phases for batching multiple kernel runs before downloading

Buffer pooling ensures that repeated evaluations of similar-sized portfolios incur no GPU memory allocation after the first call.

Data Flow: Single Evaluation

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
sequenceDiagram
    participant App as Application
    participant Adapt as PamV3Adapter
    participant Exec as PamGpuExecutor
    participant GPU as GPU Device

    App->>Adapt: Convert(contracts, riskFactors)
    Adapt->>Adapt: Schedule each contract
    Adapt->>Adapt: Flatten to SoA arrays
    Adapt-->>Exec: PamContractGpu[], PamEventGpu[], GpuMarketRate[]
    Exec->>GPU: Allocate buffers (if needed)
    Exec->>GPU: H2D: copy input arrays
    Exec->>GPU: Launch PamGpuKernel (N threads)
    GPU->>GPU: Each thread: process one contract's events
    GPU-->>Exec: D2H: copy PamEventResultGpu[]
    Exec-->>App: Results array
```

Data Flow: Monte Carlo (PayoffCube)

For scenario-based valuation, the kernel operates in 2D:

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
sequenceDiagram
    participant App as Application
    participant Exec as PayoffCubeExecutor
    participant GPU as GPU Device

    App->>Exec: Upload contracts + scenario rates
    Exec->>GPU: H2D: contracts, events, scenario rates
    Exec->>GPU: Launch PayoffCubeKernel (C×S threads)
    GPU->>GPU: Each thread: contract c, scenario s
    GPU->>GPU: Accumulate payoffs into flat cube
    GPU-->>Exec: D2H: PayoffCube[K×C×S×T]
    Exec-->>App: Aggregated results
```

Continue Reading

- Data Structures — blittable structs and SoA layout
- Kernels — PAM kernel, PayoffCube kernel, and Life kernel
- Executors — device management, buffer pooling, and profiling

Testing Overview

Testing Overview

Philosophy

The testing strategy is built on one principle: **every layer of the system must produce results identical to the ACTUS reference, to within 10 decimal places.** This is verified by running the 42 official ACTUS PAM reference test cases on every execution engine (CPU and GPU) and comparing every event's payoff and state values against the certified expected results.

The 42 Reference Tests

ACTUS publishes a comprehensive set of reference test cases for each contract type. For PAM, there are 42 tests covering:

- Fixed and floating interest rates
- Monthly, quarterly, semi-annual, and annual payment frequencies
- Different day count conventions (A/365, A/360, 30E/360, A/A ISDA)
- Business day conventions (various shift/calc combinations)
- Rate resets with caps, floors, and spread adjustments
- Fee structures (absolute and notional-based)
- Scaling adjustments
- Purchase and termination events
- Edge cases (same-day events, end-of-month handling)

Each test case defines: contract terms, observed market data (rate observations), and expected results (event date, type, payoff, and state values for every event).

Test Architecture

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    JSON["actus-tests-pam.json<br/>42 reference test cases"] --> CPU_TEST["CPU T
    JSON --> GPU_TEST["GPU Test Suite<br/>(PamTestsGpu.cs)"]
```

```
    CPU_TEST --> CPU_ENGINE["CPU Engine<br/>PrincipalAtMaturity"]
    GPU_TEST --> ADAPTER["PamV3Adapter"]
    ADAPTER --> GPU_ENGINE["GPU Engine<br/>PamGpuExecutor + PamGpuKernel"]

    CPU_ENGINE --> COMPARE_CPU["Compare vs expected<br/>Tolerance: 2×10⁻¹⁰"]
    GPU_ENGINE --> COMPARE_GPU["Compare vs expected<br/>Tolerance: 2×10⁻¹⁰"]
```

CPU Test Suite (PamTests.cs)

The CPU test suite loads the JSON test file, parses each test case into contract terms and risk factors, generates the event schedule, applies all events, and compares the computed payoff and state values against expected results.

The tolerance is 2×10^{-10} (two ten-billionths). This is tight enough to catch subtle calculation errors while accommodating the inherent limits of double-precision floating-point arithmetic.

GPU Test Suite (PamTestsGpu.cs)

The GPU test suite follows the same pattern but routes through the adapter and GPU executor:

1. Parse the same JSON test cases
2. Convert contract terms to GPU format via PamV3Adapter
3. Execute on the GPU via PamGpuExecutor
4. Filter results (exclude pre-purchase events to match reference expectations)
5. Compare against the same expected values with the same tolerance

This ensures that every transformation step (domain → GPU struct → kernel computation → result struct) preserves correctness.

Life Insurance Tests

The life insurance GPU tests verify:

- Projection output shape (correct number of time steps and channels)
- Probability conservation (probActive + probDeath + probLapsed ≤ 1.0)
- Deterministic reproducibility (same inputs produce same outputs)
- Edge cases (zero premium, immediate lapse, extreme ages)

Continuous Verification

The project enforces a "re-certify after every change" discipline: after any modification to the engine, adapter, or kernel, all 42 reference tests are re-run. Any change that causes even one test to fail — by even a fraction above the tolerance — is rejected until the root cause is fixed. This means the system is never in an incorrect state.

PAM Contract Implementation

PAM Contract Implementation

Overview

PAM (Principal at Maturity) is the most fundamental ACTUS contract type. It models any financial instrument where a principal amount is exchanged at the start, periodic interest payments occur during the life of the contract, and the full principal is returned at maturity. Common examples include bullet loans, fixed-term deposits, zero-coupon bonds, and simple floating-rate notes.

The PAM implementation is the only contract type fully realized in this codebase. It is also the most complex: the 42 official ACTUS reference test cases exercise every feature — fixed and floating rates, fee schedules, capitalization periods, scaling indices, purchase and termination events, and all convention combinations. Passing all 42 tests means the implementation is complete and correct according to the ACTUS standard.

The implementation lives in a single static class, `PrincipalAtMaturity` (346 lines), in the `ActusInsurance.Core.CPU` project. It has two public methods: `Schedule()` generates the raw events, and `Apply()` evaluates them. All supporting abstractions — contract terms, events, state, conventions — live in `ActusInsurance.Core`.

Contract Terms: The Input

Before understanding what the engine does, you need to understand what it works with. A PAM contract is fully defined by the `PamContractTerms` class — a sealed data object with over 40 properties. These properties fall into seven categories, described here from most fundamental to most specialized.

Identity and Dates

Every contract has a unique identifier, a role (asset or liability), and a set of critical dates that define its lifecycle:

Field	Type	Description
ContractID	string	Unique identifier for the contract

Field	Type	Description
ContractType	string	Always returns "PAM" (hard-coded)
ContractRole	ContractRole	RPA (asset) or RPL (liability) — determines the sign of cash flows
StatusDate	DateTime	The observation date — the "as of" date for the current state
InitialExchangeDate	DateTime	When principal is first exchanged (contract activation)
MaturityDate	DateTime	When principal is returned (contract ends)
PurchaseDate	DateTime?	Optional: date of secondary market purchase
TerminationDate	DateTime?	Optional: date of early termination
CapitalizationEndDate	DateTime?	Optional: date when interest capitalization stops

The `StatusDate` is critical: it determines the initial state. If the StatusDate is before the InitialExchangeDate, the contract has not yet started and the initial principal is zero. If it is after, the contract is active and the state reflects its current position.

Principal and Interest

The core financial parameters:

Field	Type	Default	Description
NotionalPrincipal	double	0	Face value of the contract
NominalInterestRate	double	0	Annual interest rate (e.g., 0.05 = 5%)
AccruedInterest	double	0	Interest accumulated since the last payment date
PremiumDiscountAtIED	double	0	Premium or discount applied at initial exchange
PriceAtPurchaseDate	double	0	Price paid for secondary market purchase
PriceAtTerminationDate	double	0	Price received at early termination

Payment Cycles

Periodic events (interest payments, rate resets, fees, scaling updates) are defined by cycle strings — ISO 8601-style period expressions that specify frequency and stub handling. Each cycle has an optional anchor date that determines when the cycle starts; if omitted, it defaults to the InitialExchangeDate.

Field	Type	Description
CycleOfInterestPayment	string?	Payment frequency (e.g., "P3ML0" = quarterly, long stub)
CycleAnchorDateOfInterestPayment	DateTime?	Start date of the payment cycle
CycleOfRateReset	string?	Rate reset frequency (floating-rate contracts only)
CycleOfFee	string?	Fee payment frequency
CycleOfScalingIndex	string?	Scaling index update frequency

The cycle string format is explained in detail in Schedule Generation. In brief: "P3ML0" means a period of 3 months with a long stub, and "P1YL0" means annual with a long stub.

Rate Reset Parameters

For floating-rate contracts, the interest rate is periodically updated from market data. The rate reset mechanism has six parameters that control how the new rate is derived and constrained:

Field	Type	Default	Description
MarketObjectCodeOfRateReset	string?	null	Key to look up the market rate in the RiskFactorModel
RateMultiplier	double	1.0	Multiplied by the market rate before adding the spread
RateSpread	double	0	Added to the scaled market rate
NextResetRate	double?	null	If set, used for the first RRF event instead of a market lookup
LifeCap	double	+∞	Maximum rate allowed over the contract's lifetime
LifeFloor	double	−∞	

Field	Type	Default	Description
			Minimum rate allowed over the contract's lifetime
PeriodCap	double	+∞	Maximum rate change allowed in a single reset period
PeriodFloor	double	−∞	Minimum rate change allowed in a single reset period

The rate reset algorithm (described in State Machine) applies these constraints in two stages: first clamping the delta (change) to the period bounds, then clamping the resulting absolute rate to the lifetime bounds.

Fee Parameters

Field	Type	Description
FeeBasis	string?	"A" for absolute (fixed amount per period) or "N" for notional (percentage of principal)
FeeRate	double	The fee amount (absolute) or fee rate (notional)
FeeAccrued	double	Fees accumulated since the last fee payment

Scaling Parameters

Scaling adjusts future cash flows based on an external index (e.g., an inflation index). The scaling effect can apply to the notional, to the interest, or to both:

Field	Type	Default	Description
MarketObjectCodeOfScalingIndex	string?	null	Key to look up the index value in the RiskFactorModel
ScalingIndexAtContractDealDate	double	0	Base index value at contract inception
NotionalScalingMultiplier	double	1.0	Current multiplier for notional-dependent cash flows
InterestScalingMultiplier	double	1.0	Current multiplier for interest-dependent cash flows
ScalingEffect	string?	null	"N" (notional only), "I" (interest only), or "NI" (both)

Convention Selections

Each contract specifies which financial conventions to use for date arithmetic:

Field	Type	Default	Description
DayCountConvention	enum	A_365	How year fractions are calculated (see Conventions)
BusinessDayConvention	enum	NOS	How non-business days are handled
EndOfMonthConvention	bool	false	Whether to snap schedule dates to month-ends
Calendar	enum	NC	Which calendar defines business days

The RoleSign

A key derived property: `RoleSign` returns +1 for asset positions (RPA, BUY, RFL, RF) and −1 for liability positions (RPL, SEL, PFL, PF). This single sign determines the direction of all cash flows. It is computed once and cached for performance.

Deserialization

The `FromDictionary()` static factory method creates a `PamContractTerms` from a generic `IDictionary<string, object>`, matching the JSON structure used in the official ACTUS test suite. It handles type conversions (string to double, string to DateTime, string to enum), sets defaults for optional dates, and uses invariant culture parsing throughout.

Schedule Generation

The `Schedule()` method is the first stage of the pipeline. It takes a horizon date (how far into the future to generate events) and the contract terms, and produces a list of raw, unevaluated events.

The Algorithm

Schedule generation follows a fixed eight-step process:

Step 1 — Mandatory events. Every PAM contract has at least two events: an Initial Exchange Date (IED) and a Maturity Date (MD). These are always created.

```
events.Add(IED at InitialExchangeDate)
events.Add(MD at MaturityDate)
```

Step 2 — Purchase event. If `PurchaseDate` is set, add a PRD event.

Step 3 — Interest payment schedule. If `CycleOfInterestPayment` is set (or an anchor date is provided), generate periodic dates from the anchor (or IED) to maturity using the `ScheduleFactory`. These become either IP events (interest payment) or IPCI events (interest capitalization), depending on whether the date falls before or after the `CapitalizationEndDate`:

- Dates on or before `CapitalizationEndDate` → IPCI events
- Dates after `CapitalizationEndDate` → IP events
- If `CapitalizationEndDate` is not in the generated schedule, it is added as an additional IPCI event

If there is no payment cycle but a `CapitalizationEndDate` exists, a single IPCI event is created at that date.

Step 4 — Rate reset schedule. If `CycleOfRateReset` is set, generate periodic dates and create RR events. One exception: the first reset event after the StatusDate where `NextResetRate` is available becomes an RRF event (fixed rate reset) instead.

Step 5 — Fee payment schedule. If `CycleOfFee` is set, generate periodic dates and create FP events.

Step 6 — Scaling schedule. If `ScalingEffect` contains "N" or "I" and a cycle is defined, generate periodic dates and create SC events.

Step 7 — Termination handling. If `TerminationDate` is set:

- Add an IP event at the termination date (final interest payment)
- Add a TD event at the termination date
- Remove all events that fall after the termination date

Step 8 — Filter and sort. Remove events before the StatusDate, remove events beyond the horizon date, and sort the remaining events by time (ascending) and event-type priority (for same-time events).

Business Day Adjustment

Each event carries two times:

- **ScheduleTime** — the original, unadjusted date from the cycle
- **Time** — the date shifted by the business day convention (e.g., moved to the next business day if the original falls on a weekend)

The shifted time is used for ordering and settlement. The schedule time is used for interest accrual calculations in CalcShift conventions, where the calculation should use the original contractual date regardless of when the payment actually settles.

Event Application

The `Apply()` method is the second stage. It takes the raw events from `Schedule()`, the contract terms, and a risk factor model (market data), and produces evaluated events with computed payoffs and state snapshots.

State Initialization

Before processing any events, the contract state must be initialized from the terms. This is not trivial — it depends on the relationship between the StatusDate and the InitialExchangeDate.

If the contract has not yet started (StatusDate < IED):

- NotionalPrincipal = 0
- NominalInterestRate = 0
- AccruedInterest = 0

If the contract is already active (StatusDate ≥ IED):

- NotionalPrincipal = RoleSign × NotionalPrincipal from terms
- NominalInterestRate = NominalInterestRate from terms

Accrued interest initialization follows a three-level fallback:

1. If the nominal rate is zero → accrued interest = 0
2. If the terms provide a non-zero AccruedInterest → use that value directly
3. Otherwise, compute it: find the last interest payment cycle date before StatusDate, compute the year fraction from that date to StatusDate, and multiply by rate × notional

Scaling multipliers are initialized from the terms (default 1.0).

Fee accrued follows the same fallback: use the provided value if non-zero, otherwise default to 0.

The Processing Loop

After initialization, the events are sorted and processed in sequence:

```
for each event in sorted order:
    event.Evaluate(ref stateSpace, terms, riskFactors,
businessDayAdjuster)
```

The `Evaluate()` method on each event performs three operations:

1. **ComputePayoff** — calculates the cash flow for this event type
2. **UpdateState** — modifies the state space according to this event type
3. **Snapshot** — copies the current state variables into the event for reporting

Post-Processing

After all events are evaluated, if the contract has a PurchaseDate, events before the purchase date are removed (except AD events), and the IP event on the purchase date itself is removed. This reflects the fact that the buyer does not receive interest payments that occurred before their purchase.

Payoff Formulas

Each event type computes its payoff differently. The formulas below use the following notation:

- `fxRate` — foreign exchange rate (currently always 1.0)
- `R` — RoleSign (+1 or −1)
- `N` — NotionalPrincipal (state, already signed by role)
- `r` — NominalInterestRate (state)
- `A` — AccruedInterest (state)
- `Δt` — year fraction from previous event to current event
- `S_n` — NotionalScalingMultiplier (state)
- `S_i` — InterestScalingMultiplier (state)

IED (Initial Exchange Date)

```
Payoff = fxRate × R × (−1) × (NotionalPrincipal_terms +
PremiumDiscountAtIED)
```

The negative sign reflects that the lender pays out the principal. The RoleSign ensures the correct direction for asset vs. liability positions.

MD (Maturity Date)

```
Payoff = fxRate × S_n × N
```

The principal is returned, scaled by the notional scaling multiplier if scaling is active.

IP (Interest Payment)

```
Payoff = fxRate × S_i × (A + Δt × r × N)
```

This is the most frequently computed payoff. It includes both the interest that has been accruing since the last event (`A`) and the interest for the final sub-period from the last accrual to this payment date (`Δt × r × N`). The interest scaling multiplier adjusts for index-linked contracts.

IPCI (Interest Capitalization)

```
Payoff = 0
```

No cash flow occurs. Instead, the accrued interest is added to the principal (handled in the state update).

FP (Fee Payment)

For absolute fees (FeeBasis = "A"):

```
Payoff = fxRate × R × FeeRate
```

For notional fees (FeeBasis = "N"):

```
Payoff = fxRate × (FeeAccrued + N × FeeRate × Δt)
```

PRD (Purchase Date)

```
Payoff = fxRate × R × (−1) × PriceAtPurchaseDate
```

TD (Termination Date)

```
Payoff = fxRate × R × PriceAtTerminationDate
```

RR, RRF, SC (Rate Reset, Fixed Rate Reset, Scaling)

```
Payoff = 0
```

These are administrative events — they modify the state but produce no cash flow.

Interest Accrual

Between any two consecutive events, interest accrues continuously. The engine computes this accrual before processing certain event types (IP, IPCI, RR, FP, SC). The accrual formula:

```
AccruedInterest += NominalInterestRate × NotionalPrincipal ×
YearFraction(prevDate, currentDate)
```

The year fraction depends on the day count convention and is computed using the `DayCountCalculator`. The dates used for the calculation depend on the business day convention: CalcShift conventions use the original schedule dates, while ShiftCalc conventions use the shifted dates.

Fee accrual (for notional-basis fees) follows the same pattern:

```
FeeAccrued += FeeRate × NotionalPrincipal × YearFraction(prevDate,
currentDate)
```

Both accruals only occur when the contract has a non-zero notional and the event is strictly after the previous status date.

Scaling

When a scaling event (SC) is processed, the engine looks up the current index value from the risk factor model and computes a scaling factor relative to the base index:

```
scalingFactor = currentIndex / ScalingIndexAtContractDealDate
```

Depending on the `ScalingEffect` setting:

- If it contains "N": `NotionalScalingMultiplier = scalingFactor × model.NotionalScalingMultiplier`
- If it contains "I": `InterestScalingMultiplier = scalingFactor × model.InterestScalingMultiplier`
- If it contains "NI": both are updated

This mechanism allows contracts to be linked to inflation indices or other market benchmarks, with the scaling multipliers flowing into the payoff formulas for MD (notional scaling) and IP (interest scaling).

Continue Reading

- State Machine — detailed state transitions for each event type, including the rate reset algorithm with caps and floors
- Schedule Generation — how cycle strings are parsed and periodic date sequences are built
- Conventions — day count, business day, and calendar convention implementations
- Risk Factor Model — how market data is stored and retrieved
- Technical Reference — complete listing of all types, enumerations, and interfaces

Scenario Demo — CPU vs GPU Comparison

Scenario Demo — CPU vs GPU Comparison

Overview

The ScenarioCpuGpuCalcDateDemo is a focused sample application that demonstrates one specific capability: evaluating the same portfolio on both CPU and GPU engines and verifying that the results match. It is simpler than the full CLI tool and serves as a good starting point for understanding the evaluation pipeline.

What It Demonstrates

1. **CPU/GPU parity** — the same contracts produce identical results on both engines
2. **CalcDateIndex handling** — how the system splits rates into "before" (historical) and "after" (forward/scenario-specific) segments
3. **CSV export** — writing results to files for external analysis

Components

Component	Purpose
Program.cs	Entry point — sets up contracts, scenarios, and runs both engines
CpuEngine	Evaluates contracts using the CPU PAM implementation
GpuEngine	Evaluates contracts using the GPU PAM kernel
RateScenarios	Generates or loads interest rate scenarios
CsvSink	Exports results to CSV files

Usage

The demo creates a small synthetic portfolio, generates a few scenarios, evaluates on both CPU and GPU, compares results, and exports to CSV. It is designed to be run as a quick verification that the GPU engine matches the CPU engine on the developer's local machine.

GPU Data Structures

GPU Data Structures

Overview

GPU kernels operate exclusively on blittable types — fixed-size structs containing only primitive numeric fields. This page documents the GPU data structures and the transformation rules that convert rich CPU domain objects into GPU-compatible format.

PamContractGpu

The primary contract representation on the GPU. Each contract is a single fixed-size struct containing all parameters needed for event evaluation.

Field	Type	Source
StatusDateTicks	long	StatusDate converted to ticks (0 = null)
InitialExchangeDateTicks	long	InitialExchangeDate → ticks
MaturityDateTicks	long	MaturityDate → ticks
PurchaseDateTicks	long	PurchaseDate → ticks (0 = none)
TerminationDateTicks	long	TerminationDate → ticks (0 = none)
NotionalPrincipal	double	Direct copy
NominalInterestRate	double	Direct copy
AccruedInterest	double	Direct copy
RateSpread	double	Direct copy
RateMultiplier	double	Direct copy
NextResetRate	double	Direct copy
LifeCap / LifeFloor	double	Rate cap and floor
PeriodCap / PeriodFloor	double	Per-period rate bounds

Field	Type	Source
FeeRate	double	Direct copy
FeeAccrued	double	Direct copy
FeeBasis	int	"A"=0, "N"=1
DayCountConventionCode	int	Enum → integer code
ContractRoleSign	double	+1.0 or −1.0
EventOffset / EventCount	int	Slice into shared PamEventGpu[]
MarketRateOffset / MarketRateCount	int	Slice into shared GpuMarketRate[]
Initial state fields	double	NotionalPrincipal, rates, accrued at IED

Key Design Decisions

Dates as ticks: DateTime objects cannot exist on the GPU. All dates are converted to `long` tick counts. A value of 0 indicates a null/absent date.

Enum codes: String-based enums (like "Annual" for payment frequency) are converted to integer codes during adaptation.

Event offset/count: Rather than giving each contract its own event array, all events are packed into a single flat array. Each contract stores its starting offset and event count. This eliminates per-contract memory allocation on the GPU.

PamEventGpu

Each scheduled event is represented as a fixed-size struct in the shared event array.

Field	Type	Description
ScheduleTimeTicks	long	Original scheduled date (before business day shifting)
EventTimeTicks	long	Actual event date (after business day shifting)
CalcTimeTicks	long	Calculation reference date (for CalcShift conventions)
EventType	int	GpuEventType code (IED=0, IP=1, ... CD=11)
RateIndex	int	Index into GpuMarketRate[] array (−1 if no rate lookup)

PamEventResultGpu

The output struct written by the kernel for each event.

Field	Type	Description
Payoff	double	Computed payoff amount
NotionalPrincipal	double	State: notional after this event
NominalInterestRate	double	State: interest rate after this event
AccruedInterest	double	State: accrued interest after this event
FeeAccrued	double	State: fee accrued after this event

Struct is padded to 8-byte alignment for optimal GPU memory access.

GpuMarketRate

Market rate observations for rate-reset events.

Field	Type	Description
TimeTicks	long	Observation date
Value	double	Rate value

GpuEventType Enum

Maps CPU EventType to GPU integer codes:

Code	Event Type	Description
0	IED	Initial Exchange Date
1	IP	Interest Payment
2	IPCI	Interest Payment Capitalisation
3	PRD	Purchase Date
4	TD	Termination Date
5	RR	Rate Reset (variable)

Code	Event Type	Description
6	RRF	Rate Reset (fixed)
7	FP	Fee Payment
8	SC	Scaling
9	MD	Maturity Date
10	AD	Analysis Date
11	CD	Credit Default

SoA vs AoS Layout

The GPU layer uses **Structure of Arrays** (SoA) rather than Array of Structures (AoS) for memory layout. This ensures coalesced memory access when thousands of GPU threads read the same field across different contracts.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    subgraph "AoS (Array of Structures)"
        AOS["Contract1{principal, rate, date} | Contract2{principal, rate, date}
    end

    subgraph "SoA (Structure of Arrays)"
        SOA1["principals: [c1_principal, c2_principal, c3_principal, ...]"]
        SOA2["rates: [c1_rate, c2_rate, c3_rate, ...]"]
        SOA3["dates: [c1_date, c2_date, c3_date, ...]"]
    end
```

When all threads read the `principal` field simultaneously, SoA ensures these reads hit consecutive memory addresses, maximising GPU memory bandwidth.

Running Tests

Running Tests

Prerequisites

- .NET 9.0 SDK installed
- For GPU tests: NVIDIA GPU with CUDA support (or tests will fall back to CPU backend)

Running CPU Tests

The CPU tests are in the ActusInsurance.Tests.CPU project:

```
cd Actus-Insurance.Core
dotnet test
```

This runs all 42 PAM reference tests against the CPU engine. Each test loads a case from `actus-tests-pam.json`, evaluates it, and compares against expected results.

Running GPU Tests

The GPU tests are in the ActusInsurance.Tests.GPU project:

```
cd Actus-Insurance.GPU
dotnet test
```

This runs the same 42 PAM reference tests through the GPU pipeline (adapter → executor → kernel → results) and verifies against the same expected values. It also runs the life insurance projection tests.

If no NVIDIA GPU is available, ILGPU automatically falls back to the CPU backend. The tests still pass — they just run slower.

Interpreting Results

A successful run shows all tests passing:

- 42 PAM tests × 2 engines (CPU + GPU) = 84 total PAM test executions
- Life insurance projection tests (variable count)

Any failure indicates a correctness regression. The test output shows which specific test case failed and which event produced a different value, making it straightforward to identify the root cause.

Tolerance

The comparison tolerance is **2×10^{-10}** (0.0000000002). This is:

- Tight enough to catch: rounding errors in day count fractions, event ordering mistakes, incorrect rate cap/floor application, and state update bugs
- Loose enough to accommodate: inherent IEEE 754 double-precision rounding differences between CPU and GPU arithmetic

Running Benchmarks

Performance benchmarks use BenchmarkDotNet:

```
cd Actus-Insurance.GPU
dotnet run -c Release --project src/ActusInsurance.Benchmarks
```

This produces detailed timing statistics for CPU vs GPU evaluation at various portfolio sizes, including warm-up, iteration counts, and statistical analysis.

State Machine

State Machine

Overview

Every ACTUS contract is modeled as a deterministic state machine. The contract carries a state — a set of numeric variables — that evolves over time as events are processed. Each event reads the current state, computes a cash flow (its payoff), and produces a new state. Events are processed in strict chronological order. Given the same inputs (contract terms, market data, evaluation date), the engine always produces exactly the same output.

This page describes the state machine in full detail: what the state variables are, how interest accrues between events, and exactly how each event type transforms the state.

The State Space

The state of a PAM contract at any point in time is captured by the `StateSpace` value struct. This struct is passed by reference through the event processing loop, modified in place by each event.

Using a value struct (rather than a reference type) is a deliberate design choice: structs have copy-by-value semantics, which means passing a state to a parallel evaluation thread automatically creates an isolated copy. This is essential for the GPU and Monte Carlo extensions, where thousands of scenarios evaluate the same contract in parallel without thread-safety concerns.

State Variables

Variable	Type	Description	Initial Value
NotionalPrincipal	double	Current face value, signed by contract role	0 or RoleSign × NotionalPrincipal
NominalInterestRate	double	Current annual interest rate	0 or NominalInterestRate from terms
AccruedInterest	double	Interest accumulated since the last IP event	Computed or from terms

Variable	Type	Description	Initial Value
FeeAccrued	double	Fees accumulated since the last FP event	0 or from terms
NotionalScalingMultiplier	double	Multiplier for notional-dependent cash flows	1.0 from terms
InterestScalingMultiplier	double	Multiplier for interest-dependent cash flows	1.0 from terms
StatusDate	DateTime	Date of the current state (advances with each event)	StatusDate from terms
ContractPerformance	string?	Performance status (performing, delayed, default, etc.)	From terms

The first four variables — principal, rate, accrued interest, and accrued fees — are the "working" state that changes with most events. The scaling multipliers only change on SC events. The StatusDate advances to the schedule time of each event as it is processed.

Interest Accrual

Interest accrual is the engine's most frequently executed calculation. It happens before certain events are processed, updating the AccruedInterest (and optionally FeeAccrued) to reflect the time that has passed since the last event.

When Accrual Occurs

Accrual is triggered before these event types: IP, IPCI, RR, RRF, FP, and SC. It is not triggered before IED (the contract does not yet exist), MD (accrual is handled by the IP formula on the same date), PRD (purchase does not affect accrual), or TD (handled similarly to MD).

The Accrual Formula

```
AccruedInterest += NominalInterestRate × NotionalPrincipal ×
YearFraction(prevDate, currentDate)
```

Where:

- `prevDate` is the StatusDate (the date of the previous event)

- `currentDate` is the ScheduleTime of the current event
- `YearFraction` is computed using the contract's day count convention

Accrual only occurs when two conditions are met:

1. The StatusDate is strictly before the current event's ScheduleTime (no zero-length accrual)
2. The NotionalPrincipal is non-zero (no accrual on an inactive contract)

Fee Accrual

For notional-basis fees (FeeBasis = "N"), fee accrual follows the same pattern:

```
FeeAccrued += FeeRate × NotionalPrincipal × YearFraction(prevDate,
currentDate)
```

Absolute fees (FeeBasis = "A") do not accrue — they are paid as a fixed amount per period.

Date Selection for Calculations

The dates used in the year fraction calculation depend on the business day convention's Calc/Shift strategy:

- **CalcShift conventions** (CSF, CSMF, CSP, CSMP): calculations use the original, unshifted schedule dates. The `ShiftCalcTime()` method returns the original date.
- **ShiftCalc conventions** (SCF, SCMF, SCP, SCMP): calculations use the business-day-adjusted dates. The `ShiftCalcTime()` method returns the shifted date.
- **NOS** (No Shift): no adjustment — both dates are the same.

This distinction matters when an event date falls on a weekend. A CalcShift convention says "calculate interest as if the payment happened on the scheduled date, but actually settle on the next business day." A ShiftCalc convention says "the payment date is the business day, and interest accrues to that date."

Event Evaluation: The Three-Step Process

Every event's `Evaluate()` method follows the same three-step pattern:

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    A["Step 1: ComputePayoff"] --> B["Step 2: UpdateState"]
    B --> C["Step 3: Snapshot"]
```

Step 1 — ComputePayoff: Calculate the cash flow for this event type. The payoff depends on the event type and the current state (see PAM Contract for all formulas).

Step 2 — UpdateState: Modify the state space according to this event type's transition rules. This is where interest accrual happens (for applicable event types), followed by the type-specific state change.

Step 3 — Snapshot: Copy the four key state variables (NotionalPrincipal, NominalInterestRate, AccruedInterest, FeeAccrued) from the state space into the event object. This creates a permanent record of the state after this event, which appears in the output.

State Transitions by Event Type

IED — Initial Exchange Date

The contract begins. The principal is established and the interest rate is set.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph LR
    Before["Principal = 0<br/>Rate = 0<br/>Accrued = 0"] -->|IED| After["Princi
```

State changes:

- `NotionalPrincipal` = `RoleSign × model.NotionalPrincipal`
- `NominalInterestRate` = `model.NominalInterestRate`
- If the cycle anchor date is before the IED, accrued interest is computed from the anchor to the IED using the day count convention. This handles the case where interest starts accruing before the principal is exchanged.

Payoff:

```
fxRate × RoleSign × (−1) × (NotionalPrincipal + PremiumDiscountAtIED)
```

IP — Interest Payment

Accrued interest is paid out and the accrual counter resets to zero.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph LR
    Before["Accrued = A"] -->|IP| After["Accrued = 0"]
```

State changes:

- Interest accrual is computed (adding to AccruedInterest) before the state update
- `AccruedInterest` = 0 (reset after payment)
- All other state variables unchanged

Payoff:

```
fxRate × InterestScalingMultiplier × (AccruedInterest + Δt × Rate ×
Principal)
```

The payoff includes both the previously accrued amount and the interest for the final sub-period. The scaling multiplier adjusts for inflation-linked contracts.

IPCI — Interest Capitalization

Instead of paying interest out as cash, the accrued interest is added to the principal. This increases the base for future interest calculations (compound interest).

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph LR
    Before["Principal = P<br/>Accrued = A"] -->|IPCI| After["Principal = P + A<
```

State changes:

- Interest accrual is computed before the state update
- `NotionalPrincipal` += `AccruedInterest`
- `AccruedInterest` = 0

Payoff: 0 (no cash flow — the interest stays inside the contract)

RR — Rate Reset (Market-Driven)

The interest rate is updated from market data. The new rate is derived from an external market rate, adjusted by a multiplier and spread, and constrained by period-level and lifetime caps and floors.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph LR
    Before["Rate = R_old"] -->|RR| After["Rate = R_new<br/>(constrained)"]
```

The rate reset algorithm has four steps:

1. **Look up the market rate:** Query the RiskFactorModel using the `MarketObjectCodeOfRateReset` and the event time.

2. **Apply multiplier and spread:**

   ```
   candidateRate = marketRate × RateMultiplier + RateSpread
   ```

3. **Apply period constraints:** Compute the change from the current rate and clamp it:

   ```
   deltaRate = candidateRate − currentRate
   deltaRate = clamp(deltaRate, PeriodFloor, PeriodCap)
   newRate = currentRate + deltaRate
   ```

4. **Apply lifetime constraints:** Clamp the absolute rate:

   ```
   newRate = clamp(newRate, LifeFloor, LifeCap)
   ```

State changes:

- Interest accrual is computed before the rate change
- `NominalInterestRate` = constrained new rate

Payoff: 0 (administrative event — no cash flow)

Why two-stage clamping? Period caps/floors limit how much the rate can change in a single reset — protecting borrowers from sudden rate spikes. Lifetime caps/floors set absolute bounds over the contract's entire duration. Both constraints are standard in floating-rate loan agreements.

RRF — Rate Reset Fixed

A variant of rate reset where the new rate comes from the contract terms (`NextResetRate`) rather than market data. This is used for the first reset event after the StatusDate when a fixed next rate has been pre-agreed.

State changes: Same as RR, but using `model.NextResetRate` instead of a market lookup. Period and lifetime caps/floors still apply.

Payoff: 0

MD — Maturity Date

The contract ends. The principal is returned and all state variables are zeroed.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph LR
    Before["Principal = P<br/>Accrued = A"] -->|MD| After["Principal = 0<br/>Acc
```

State changes:

- `NotionalPrincipal` = 0
- `AccruedInterest` = 0

Payoff:

```
fxRate × NotionalScalingMultiplier × NotionalPrincipal
```

TD — Termination Date

Early contract termination. Functionally similar to maturity, but the payoff is the agreed termination price rather than the notional.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph LR
    Before["Principal = P"] -->|TD| After["Principal = 0<br/>Accrued = 0"]
```

State changes:

- `NotionalPrincipal` = 0
- `AccruedInterest` = 0

Payoff:

```
fxRate × RoleSign × PriceAtTerminationDate
```

PRD — Purchase Date

A secondary market purchase. The buyer pays the purchase price. No state change occurs — the contract's terms continue unchanged.

State changes: None

Payoff:

```
fxRate × RoleSign × (−1) × PriceAtPurchaseDate
```

FP — Fee Payment

Accumulated fees are paid out and the fee accrual counter resets.

State changes:

- Fee accrual is computed before the state update
- `FeeAccrued` = 0

Payoff (absolute basis):

```
fxRate × RoleSign × FeeRate
```

Payoff (notional basis):

```
fxRate × (FeeAccrued + NotionalPrincipal × FeeRate × Δt)
```

SC — Scaling

Scaling multipliers are updated from an external index. This changes the magnitude of future notional-dependent and interest-dependent cash flows.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph LR
    Before["S_n = old<br/>S_i = old"] -->|SC| After["S_n = factor × base_n<br/>S
```

The scaling algorithm:

```
scalingFactor = currentIndex / ScalingIndexAtContractDealDate
```

Then, depending on the `ScalingEffect`:

- If it contains "N": `NotionalScalingMultiplier = scalingFactor × model.NotionalScalingMultiplier`

- If it contains "I": `InterestScalingMultiplier = scalingFactor × model.InterestScalingMultiplier`

State changes:

- Interest accrual is computed before the scaling update
- Scaling multipliers updated as described above

Payoff: 0 (administrative event)

Event Priority

When multiple events occur on the same date, they must be processed in a deterministic order. The engine assigns a priority number to each event type — lower numbers are processed first:

Priority	Event Type	Rationale
0	IED	Contract must be activated before anything else
1	IP	Interest is paid at the old rate before any rate change
2	IPCI	Interest is capitalized at the old rate before any rate change
3	PRD	Purchase is recorded after interest events
4	TD	Termination happens after interest settlement
5	RR	Rate reset takes effect after the old rate's payments
6	RRF	Fixed rate reset, same ordering as RR
7	FP	Fee payment follows rate changes
8	SC	Scaling updates follow all payment events
9	MD	Maturity is last — principal is returned after all other processing
10	CD	Credit default is the final event type

This ordering ensures correct behavior. For example, on a date where both an IP and an RR event occur, the interest is paid at the old rate (IP at priority 1) before the rate changes (RR at priority 5). Similarly, on the maturity date, any final interest payment (IP at priority 1) is processed before the principal return (MD at priority 9).

The Complete Processing Flow

Putting it all together, here is the complete sequence for evaluating a PAM contract:

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    TERMS["Contract Terms"] --> INIT["Initialize StateSpace<br/>(principal, rat
    TERMS --> SCHED["Generate Schedule<br/>(all event types)"]
    SCHED --> SORT["Sort Events<br/>(time, then priority)"]
    SORT --> LOOP{"More Events?"}
    LOOP -->|Yes| ACCRUE["Accrue Interest & Fees<br/>(if applicable event type)
    ACCRUE --> DATE["Update StatusDate<br/>to ScheduleTime"]
    DATE --> PAYOFF["Compute Payoff<br/>(type-specific formula)"]
    PAYOFF --> UPDATE["Update State<br/>(type-specific transition)"]
    UPDATE --> SNAP["Snapshot State<br/>into Event"]
    SNAP --> LOOP
    LOOP -->|No| POST["Post-Process<br/>(remove pre-purchase events)"]
    POST --> RESULT["Return Evaluated Events"]
```

Continue Reading

- PAM Contract — contract terms and schedule generation details
- Conventions — how day count and business day conventions affect accrual
- Schedule Generation — how event dates are generated from cycle strings
- Technical Reference — complete listing of all types and enumerations

GPU Kernels

GPU Kernels

Overview

A kernel is a function that runs on the GPU with one thread per work item. This project has three main kernels, each serving a different evaluation mode.

PAM Kernel (PamGpuKernel)

The foundational kernel. Each thread processes one contract through its full event schedule.

Thread Assignment

Each GPU thread receives an Index1D corresponding to its contract index. The thread reads its contract's EventOffset and EventCount to locate its events in the shared event array, then processes them sequentially.

Per-Event Processing

For each event, the kernel:

1. Computes the year fraction between the previous event and the current event using the contract's day count convention
2. Accrues interest for the elapsed period
3. Computes the payoff based on the event type
4. Updates the contract state (notional, rate, accrued amounts)
5. Writes the result to the output array

Day Count Conventions on GPU

The kernel implements five day count conventions directly in GPU code:

Convention	Year Fraction Calculation
A/365 (Actual/365 Fixed)	actual days / 365
A/360 (Actual/360)	actual days / 360

Convention	Year Fraction Calculation
30E/360	(360 × Δyear + 30 × Δmonth + Δday) / 360
A/336	actual days / 336
AA (Actual/Actual ISDA)	days-in-year-1 / 365-or-366 + days-in-year-2 / 365-or-366

Date arithmetic is performed entirely in ticks with hour-level rounding to maintain numerical stability.

Event Type Handlers

Each event type has specific payoff and state-update logic:

Event	Payoff	State Update
IED	−roleSign × notional × (1 − premiumDiscount)	Set initial notional and rates
IP	roleSign × (accrued + timeAccrued) × interestScaling	Reset accrued to 0
IPCI	0	Add accrued to notional
RR	0	Update rate from market data (with cap/floor)
RRF	0	Set rate to fixed reset value
FP	roleSign × feeAmount	Reset fee accrued
SC	0	Update scaling multipliers
MD	roleSign × notional × notionalScaling	Final settlement
PRD	−roleSign × (notional + accrued) × scaling	Purchase transfer
TD	roleSign × (notional + accrued) × scaling	Termination transfer

PayoffCube Kernel

Extends the PAM kernel for Monte Carlo scenario evaluation with 2D thread indexing.

Thread Assignment

Each thread receives an Index2D(contractIndex, scenarioIndex). The thread evaluates one contract under one scenario, using scenario-specific interest rates for rate-reset events.

Scenario Rate Handling

Rates are split into two arrays: "prior" (before the calculation date) and "after" (from the calculation date forward). A calcDateIndex boundary determines which array a rate-reset event reads from. This allows the same historical rates to be shared across all scenarios while only the forward rates vary.

Output Accumulation

Instead of writing per-event results, the PayoffCube kernel accumulates payoffs into a flat output cube indexed by [kind × contract × scenario × timeStep]. The kernel uses atomic addition to safely accumulate when multiple events map to the same time step.

Cube Dimensions

Dimension	Description
K (kind)	Payoff kind (currently 1: total payoff)
C (contracts)	Number of contracts
S (scenarios)	Number of Monte Carlo paths
T (time steps)	Number of discrete time grid points

Life Insurance Projection Kernel

A fundamentally different kernel that projects insurance policies through a Markov state transition model.

Thread Assignment

Each thread processes one policy over a multi-year projection horizon (typically 30 years), stepping through monthly time increments.

Per-Time-Step Processing

At each time step, the kernel:

1. Looks up age-adjusted mortality rate from the mortality table (Gompertz-Makeham model)

2. Applies smoker loading (1.35× for smokers) and underwriting loading
3. Looks up duration-adjusted lapse rate from the lapse table
4. Looks up disability incidence from the disability table
5. Applies competing-risk decrements (total capped at 1.0)
6. Computes expected cash flow: premium inflow − death benefit outflow
7. Updates state probabilities (probability active, death-absorbed, lapse-absorbed)

Output Structure

Each time step produces a four-channel output:

Channel	Description
ExpectedCashflow	Probability-weighted net cash flow
ProbActive	Probability the policy is still active
ProbDeathClaimed	Cumulative probability of death claim
ProbLapsed	Cumulative probability of lapse

Lookup Tables

Pre-built on CPU and uploaded to GPU as flat arrays:

Table	Dimensions	Source
Mortality	61 ages × 3 genders = 183 floats	Gompertz-Makeham parametric model
Lapse	31 durations × 3 premium modes = 93 floats	Duration-dependent base rates
Disability	61 ages × 3 genders = 183 floats	Age-dependent incidence rates
Markov hazards	10 × 10 = 100 ints	State transition hazard type matrix

GPU Executors

GPU Executors

Overview

Executors manage the full lifecycle of GPU execution: device selection, buffer allocation, data transfer, kernel dispatch, and result retrieval. They encapsulate all ILGPU-specific code, presenting a clean API to the rest of the system.

PamGpuExecutor

The primary executor for single-batch PAM contract evaluation.

Accelerator Selection

The factory method selects the best available device:

1. **CUDA** — highest performance on NVIDIA GPUs
2. **OpenCL** — cross-vendor fallback
3. **CPU** — always available, slowest but functional

Selection happens once at construction time. The chosen accelerator is reused for all subsequent evaluations.

Execution Modes

EvaluateBatch — all-in-one pipeline:

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph LR
    A["Allocate<br/>buffers"] --> B["H2D<br/>transfer"]
    B --> C["Launch<br/>kernel"]
    C --> D["Synchronise"]
    D --> E["D2H<br/>transfer"]
    E --> F["Return<br/>results"]
```

UploadAndExecute + DownloadResults — split phases for batching:

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph LR
    A["Upload +<br/>Execute"] --> B["Upload +<br/>Execute"]
    B --> C["Upload +<br/>Execute"]
    C --> D["Download<br/>all results"]
```

This split mode is used by the Monte Carlo pipeline to submit multiple scenario batches before downloading.

Buffer Pooling

GPU memory allocation is expensive. The executor pools buffers to avoid repeated allocation:

1. On first call, buffers are allocated to match the input size
2. If a subsequent call needs more space, buffers are reallocated at 1.5× the required size + 16 elements of padding
3. Buffers never shrink — the high-water mark persists
4. All buffers are disposed when the executor is disposed

This means the common case (repeated evaluation of similar-sized portfolios) incurs zero GPU memory allocation after the first call.

Kernel Caching

The ILGPU kernel is compiled from C# to GPU instructions on first use. This compilation takes hundreds of milliseconds. The compiled kernel is cached and reused for all subsequent calls, making repeat evaluations fast.

PayoffCubeExecutor

Extends the execution model for Monte Carlo scenario evaluation with 2D kernels.

Scenario Batching

For large scenario counts, the executor batches scenarios to fit within GPU memory. Each batch uploads a subset of scenario rates, executes the 2D kernel, and accumulates results into the payoff cube.

Output Cube Management

The output cube is a flat array of size K × C × S × T. The executor allocates this on the GPU, the kernel writes to it via atomic accumulation, and the executor downloads the complete cube after all scenario batches are processed.

Life Insurance Executor

Manages the life insurance projection pipeline with its specialised data requirements.

Lookup Table Upload

Before kernel execution, the executor uploads the pre-built lookup tables (mortality, lapse, disability, Markov hazards) to GPU memory. These tables are shared across all policy threads and remain resident for the duration of the evaluation.

Scenario Replication

For life insurance Monte Carlo, policies are replicated C×S times before kernel dispatch. Each replica carries per-scenario adjustments (e.g., different mortality stress factors). This replication happens on the CPU before upload, and the kernel processes each replica as an independent thread.

Profiling

The GPU profiler (enabled via the `ACTUS_GPU_PROFILE=1` environment variable) instruments the execution pipeline to measure time spent in each phase: buffer allocation, H2D transfer, kernel execution, synchronisation, and D2H transfer. This helps identify bottlenecks and verify that the kernel execution dominates over transfer overhead for large portfolios.

Schedule Generation

Schedule Generation

Overview

Schedule generation is the process of turning a contract's cycle definitions (e.g., "pay interest every 3 months") into a concrete list of dates. These dates become events in the contract's timeline. The schedule generator is used by the PAM contract processor for all periodic event types: interest payments (IP/IPCI), rate resets (RR/RRF), fee payments (FP), and scaling updates (SC).

The schedule generation pipeline has three components:

1. **Cycle parsing** — converting a cycle string like "P3ML0" into its constituent parts (period, stub type)
2. **Date sequence generation** — the `ScheduleFactory` that produces periodic dates from a start date to an end date
3. **Date adjustment** — applying end-of-month and business day conventions to the generated dates

Cycle Strings

A cycle string defines how often an event recurs and how to handle partial periods at the boundaries. The format follows an extended ISO 8601 convention:

```
P[value][unit]L[stub]
```

Where:

- `P` — marks this as a period definition
- `[value]` — the number of units per cycle (e.g., 3)
- `[unit]` — the time unit: `Y` (years), `M` (months), `D` (days)
- `L` — separator before the stub indicator
- `[stub]` — how to handle partial periods: `0` (long stub) or `1` (short stub)

Common Cycle Strings

Cycle String	Meaning
P1ML0	Every 1 month, long stub
P3ML0	Every 3 months (quarterly), long stub
P6ML0	Every 6 months (semi-annual), long stub
P1YL0	Every 1 year (annual), long stub
P1DL0	Every 1 day (daily), long stub
P3ML1	Every 3 months, short stub

What is a Stub?

A stub is a partial period that occurs when the schedule's start date and end date don't align perfectly with the cycle length. For example, a quarterly cycle starting on January 15 and ending on November 30 has a partial period at the end (October 15 to November 30 — only 46 days instead of a full quarter).

Long stub (L0): The partial period is merged into the adjacent regular period, creating one longer-than-normal period. In the example above, the last period would run from July 15 to November 30 — longer than a normal quarter. The schedule removes the second-to-last generated date to create this merged period.

Short stub (L1): The partial period stands on its own as a short period. The last period runs from October 15 to November 30, and the prior period runs from July 15 to October 15 (a full quarter).

Cycle Parsing

The `CycleUtils` class parses cycle strings into their components:

- `IsPeriod(cycle)` — checks if the string starts with "P" (period-based cycle vs. weekday-based)
- `ParsePeriod(cycle)` — returns a `Period` with Years, Months, and Days fields
- `ParseStub(cycle)` — returns the stub character: '0' (long) or '1' (short)

The `Period` class has a `GetMonths()` method that converts years to months: `Months + (Years × 12)`. This is used by the end-of-month adjuster to determine whether EOM logic applies (only for month-based periods).

The ScheduleFactory

The `ScheduleFactory.CreateSchedule()` method is the core schedule generator. It takes five parameters:

Parameter	Type	Description
startTime	DateTime	Cycle anchor — the first date in the sequence
endTime	DateTime	Cycle end — the date the sequence runs to
cycle	string	The cycle string (e.g., "P3ML0")
endOfMonthConvention	bool	Whether to apply EOM adjustment
includeEndDate	bool	Whether to include the end date in the output

The Algorithm

Step 1 — Handle null cycles. If no cycle is provided, return just the start date (and optionally the end date). This handles single-event schedules.

Step 2 — Parse the cycle. Extract the stub type, the period (Years/Months/Days), and create an `EndOfMonthAdjuster` based on the convention flag, the start date, and the cycle. The EOM adjuster only activates if the start date is the last day of its month and the cycle is month-based.

Step 3 — Generate dates forward. Starting from `startTime`, repeatedly add the period to produce dates:

```
counter = 1
currentDate = startTime
while currentDate < endTime:
    add currentDate to schedule
    if period.Years > 0:
        nextDate = startTime + (Years × counter)
    else if period.Months > 0:
        nextDate = startTime + (Months × counter)
    else:
        nextDate = startTime + (Days × counter)
    currentDate = EndOfMonthAdjuster.Shift(nextDate)
    counter++
```

Note that each date is computed from the original start date (not from the previous date). This avoids cumulative rounding errors that would occur with repeated addition.

Step 4 — Add the end date if `includeEndDate` is true.

Step 5 — Handle long stubs. If the stub type is "long" and the generated schedule has more than two dates, and the last generated date does not exactly equal the end date, remove the second-to-last date. This merges the final short period into the previous one, creating a single long period.

Step 6 — Return the set of dates. The schedule is stored as a `HashSet<DateTime>` to prevent duplicates (which can occur when a generated date coincidentally equals the end date).

Example: Quarterly Schedule

Contract starts January 1, 2025. Maturity March 15, 2026. Cycle: "P3ML0" (quarterly, long stub).

Generated sequence:

1. Jan 1, 2025
2. Apr 1, 2025
3. Jul 1, 2025
4. Oct 1, 2025
5. Jan 1, 2026

End date (Mar 15, 2026) is added. The last generated date (Jan 1, 2026) does not equal the end date, and stub is long, so Jan 1, 2026 is removed. Final schedule:

- Jan 1, 2025
- Apr 1, 2025
- Jul 1, 2025
- Oct 1, 2025
- Mar 15, 2026 ← long stub period (5.5 months instead of 3)

End-of-Month Adjustment

When schedule dates are generated, the `EndOfMonthAdjuster` may shift each generated date to the last day of its month. This only happens when two conditions are met:

1. The cycle anchor date is the last day of its month (e.g., January 31, February 28)
2. The cycle is month-based (the period's `GetMonths()` returns a value greater than 0)

If both conditions are met and the EOM convention is enabled, the `EndOfMonth` shift snaps each date to the last day of its month:

```
Shift(date) → DateTime(date.Year, date.Month, DaysInMonth(date.Year,
date.Month))
```

This means a monthly cycle starting on January 31 produces: Jan 31, Feb 28 (or 29), Mar 31, Apr 30, and so on — always the last day of the month.

If either condition is not met, or if the EOM convention is disabled, the `SameDay` shift returns the date unchanged.

Weekday-Based Cycles

Not all cycles are period-based. Some contracts specify events on specific weekdays — for example, "the second Monday of every month." The `CycleAdjuster` class detects whether a cycle string represents a period or a weekday pattern and dispatches to the correct implementation.

Period Cycle Adjuster

For period-based cycles (strings starting with "P"), the `PeriodCycleAdjuster` simply adds or subtracts the period as a `TimeSpan`:

```
PlusCycle(date) → date + period
MinusCycle(date) → date − period
```

Weekday Cycle Adjuster

For weekday-based cycles, the string format is `[position][weekday]` — for example, "2Mon" means the second Monday. The `WeekdayCycleAdjuster` finds the nth occurrence of the target weekday in the next month:

```
PlusCycle(date):
    targetMonth = date.AddMonths(1)
    find the [position]th [weekday] in targetMonth
    if result is in the next month (overflowed), use (position-1)th
instead
```

The algorithm finds the first occurrence of the target weekday in the month, then adds (position − 1) × 7 days. If the result falls into the next month (because the month doesn't have enough of that weekday), it backs up by one week.

How Schedule Dates Become Events

The `ScheduleFactory` produces raw dates. The PAM contract processor (`PrincipalAtMaturity.Schedule()`) converts these dates into typed events:

1. Generate dates using `ScheduleFactory.CreateSchedule(anchor, maturity, cycle, eom, includeEnd)`
2. For each date, create a `ContractEvent` with:
   - `ScheduleTime` = the raw generated date
   - `Time` = `BusinessDayAdjuster.ShiftEventTime(date)` — the business-day-adjusted date
   - `Type` = the appropriate event type (IP, RR, FP, SC, etc.)
   - `Currency` = the contract's currency

The two-time design (ScheduleTime vs. Time) is critical: the ScheduleTime preserves the contractual date for calculation purposes, while Time reflects the actual settlement date after business day adjustment.

Time Utilities

TimeAdjuster

A utility class that rounds timestamps to the nearest full hour:

```
if minutes < 30: round down to current hour
else: round up to next hour
```

This is used to normalize timestamps that may have been stored with sub-hour precision.

Constants

The engine defines maximum contract lifetimes to prevent unbounded schedule generation:

Constant	Value	Description
MAX_LIFETIME	50 years	Maximum duration for most contract types
MAX_LIFETIME_STK	10 years	Maximum for stock-type contracts
MAX_LIFETIME_UMP	10 years	Maximum for undefined maturity profiles

Continue Reading

- PAM Contract — how the schedule is generated in the full contract pipeline
- Conventions — end-of-month and business day conventions that affect schedule dates
- State Machine — how the generated events are evaluated
- Technical Reference — Period class, CycleUtils, and ScheduleFactory details

Risk Factor Model

Risk Factor Model

Overview

The risk factor model is the engine's interface to the outside world. While contract terms and event logic are fully deterministic, certain events need external data — a current market interest rate for a rate reset, or a current index value for a scaling update. The `RiskFactorModel` class provides this data.

The model is deliberately simple: it stores rates keyed by a market object code (a string identifier) and a timestamp, and provides a single lookup method. It does not model interest rate dynamics, generate scenarios, or forecast future rates. Those responsibilities belong to higher-level components (the Monte Carlo simulation layer, the Vasicek rate model). The risk factor model is purely a lookup service.

Concepts

Market Object Code

A market object code is a string that uniquely identifies a market observable — such as an interest rate index, an inflation index, or a foreign exchange rate. In the ACTUS test suite, codes like "LIBOR_6M" or "EURIBOR_3M" are typical. The contract terms reference these codes (e.g., `MarketObjectCodeOfRateReset = "LIBOR_6M"`), and the risk factor model stores the corresponding values.

Last Observation Carried Forward (LOCF)

Market data is not available at every point in time. When the engine needs a rate for a specific date and that exact date is not in the time series, the model uses LOCF: it returns the most recent observation on or before the requested date. This is the standard approach in financial time series — the assumption is that the last known rate remains valid until a new observation arrives.

Storage

The `RiskFactorModel` maintains two parallel storage mechanisms:

Constant Rates

```
Dictionary<string, double> _constantRates
```

A single value per market object code, valid at all times. This is the fastest lookup path and handles the common case where a rate does not change over the contract's life.

Time Series Rates

```
Dictionary<string, Dictionary<DateTime, double>> _rates
```

Multiple values per market object code, each keyed by a date. This handles rates that change over time — for example, a rate reset schedule where different dates have different market rates.

Adding Data

Two methods populate the model:

`AddConstantRate(marketObjectCode, value)` — sets a single rate for all times. If a constant rate already exists for that code, it is overwritten.

`AddRate(marketObjectCode, time, value)` — adds a time-stamped rate. Multiple calls with the same code but different times build up a time series. If a rate already exists for the same code and time, it is overwritten.

Both methods can be used for the same market object code. When both exist, the constant rate takes precedence in lookups (see below).

Lookup

The `GetRate(marketObjectCode, time)` method retrieves a rate value. It follows a four-step priority chain:

Step 1 — Check constant rates. If a constant rate exists for the given code, return it immediately. This is the fastest path and covers the most common case.

Step 2 — Check exact time match. If a time series exists for the code, look for the exact requested date. If found, return that value.

Step 3 — LOCF fallback. If no exact match exists in the time series, scan all entries to find the most recent date that is on or before the requested date. Return that value.

Step 4 — Default. If no data exists at all (no constant rate, no time series, or no observations before the requested date), return 0.0.

Lookup Priority Diagram

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    REQ["GetRate(code, time)"] --> CONST{"Constant rate<br/>exists?"}
    CONST -->|Yes| RET1["Return constant"]
    CONST -->|No| TS{"Time series<br/>exists?"}
    TS -->|No| DEF["Return 0.0"]
    TS -->|Yes| EXACT{"Exact date<br/>match?"}
    EXACT -->|Yes| RET2["Return exact value"]
    EXACT -->|No| LOCF{"Find most recent<br/>date ≤ requested"}
    LOCF -->|Found| RET3["Return LOCF value"]
    LOCF -->|Not found| DEF
```

LOCF Example

Given a time series for "LIBOR_6M":

Date	Value
2025-01-01	0.030
2025-04-01	0.035
2025-07-01	0.032

Lookups:

- `GetRate("LIBOR_6M", 2025-01-01)` → 0.030 (exact match)
- `GetRate("LIBOR_6M", 2025-03-15)` → 0.030 (LOCF from Jan 1)
- `GetRate("LIBOR_6M", 2025-04-01)` → 0.035 (exact match)
- `GetRate("LIBOR_6M", 2025-05-15)` → 0.035 (LOCF from Apr 1)
- `GetRate("LIBOR_6M", 2024-12-15)` → 0.0 (no observation on or before)

Usage in the Engine

The risk factor model is used in three contexts:

Rate Resets (RR Events)

When a rate reset event is processed, the engine calls:

```
marketRate = riskFactors.GetRate(model.MarketObjectCodeOfRateReset,
eventTime)
```

This market rate is then transformed by the rate reset algorithm (multiplier, spread, caps, floors) to produce the new nominal interest rate.

Fixed Rate Resets (RRF Events)

RRF events use the `NextResetRate` from the contract terms instead of a market lookup. However, if `NextResetRate` is not set, the engine falls back to a market lookup using the same mechanism as RR events.

Scaling Updates (SC Events)

When a scaling event is processed, the engine calls:

```
scalingIndex = riskFactors.GetRate(model.MarketObjectCodeOfScalingIndex,
eventTime)
```

The returned index value is divided by the base index (`ScalingIndexAtContractDealDate`) to compute the scaling factor.

Foreign Exchange Rates

The payoff formulas include an `fxRate` term, which is obtained via:

```
fxRate = riskFactors.GetRate(model.Currency, eventTime)
```

In the current implementation, this always returns 1.0 (no multi-currency support). The infrastructure is in place for future extension.

Test Data Format

The ACTUS reference test suite provides market data in a specific JSON format:

```
{
    "dataObserved": {
        "market_index_code": {
```

```
            "marketObjectCode": "LIBOR_6M",
            "data": [
                { "timestamp": "2025-01-01", "value": "0.030" },
                { "timestamp": "2025-04-01", "value": "0.035" }
            ]
        }
    }
}
```

The test harness populates the `RiskFactorModel` by iterating over each data set and calling `AddRate()` for each timestamp/value pair. This structure allows multiple market object codes, each with their own time series.

Design Notes

The risk factor model is intentionally minimal. It does not:

- Generate future rate scenarios (that is the Monte Carlo layer's job)
- Interpolate between observations (LOCF is the only strategy)
- Validate that rates are reasonable (negative rates are valid in some markets)
- Support complex term structures (each code is a flat time series)

This simplicity is by design. The core engine's job is deterministic evaluation: given inputs, produce outputs. The risk factor model is one of those inputs — how it gets populated is the responsibility of the caller. For the 42 reference tests, the test harness populates it from JSON. For Monte Carlo simulation, the Vasicek rate model populates it with simulated rate paths. For GPU batch evaluation, the executor populates it from scenario data.

Continue Reading

- State Machine — how rate resets and scaling updates use the risk factor model
- PAM Contract — the full contract evaluation pipeline
- Technical Reference — RiskFactorModel class details

Technical Reference

Technical Reference

This page provides a comprehensive listing of every type in the ACTUS core engine. It is organized by category (enumerations, interfaces, classes, structs) and serves as a quick reference for understanding the codebase. Each entry includes the type's purpose, its members, and where it is used.

Enumerations

EventType

Defines the type of each contract event. Used in `ContractEvent.Type` and in the payoff/state-update dispatch logic.

Value	Name	Priority	Description
AD	Analysis Date	—	Placeholder for analysis (not actively used in PAM)
IED	Initial Exchange Date	0	Contract activation — principal is exchanged
MD	Maturity Date	9	Contract ends — principal is returned
IP	Interest Payment	1	Periodic interest payment
IPCI	Interest Capitalization	2	Interest added to principal instead of paid out
PRD	Purchase Date	3	Secondary market purchase
TD	Termination Date	4	Early contract termination
RR	Rate Reset	5	Interest rate updated from market data
RRF	Rate Reset Fixed	6	Interest rate updated to pre-agreed fixed value
FP	Fee Payment	7	Periodic fee payment
SC	Scaling	8	Scaling multipliers updated from index

Value	Name	Priority	Description
CD	Credit Default	10	Credit default event (not implemented in PAM)

Priority determines processing order when multiple events fall on the same date. Lower priority numbers are processed first.

ContractRole

Determines the direction of cash flows. Used in `PamContractTerms.ContractRole` and the `RoleSign` computation.

Value	Name	RoleSign	Description
RPA	Real Position Asset	+1	Receives principal (lender, depositor)
RPL	Real Position Liability	−1	Pays principal (borrower)
BUY	Buy	+1	Buyer position
SEL	Sell	−1	Seller position
RFL	Receive First Leg	+1	Swap: receives the first leg
PFL	Pay First Leg	−1	Swap: pays the first leg
RF	Receive Fixed	+1	Swap: receives fixed rate
PF	Pay Fixed	−1	Swap: pays fixed rate

DayCountConvention

Defines how year fractions are calculated for interest accrual. Used in `PamContractTerms.DayCountConvention`.

Value	String Key	Year Basis	Description
A_AISDA	"AA"	365/366	Actual/Actual ISDA — leap-year-aware
A_360	"A360"	360	Actual/360 — actual days ÷ 360
A_365	"A365"	365	Actual/365 Fixed — actual days ÷ 365
E30_360ISDA	"30E360ISDA"	360	30E/360 ISDA — with maturity date exception

Value	String Key	Year Basis	Description
E30_360	"30E360"	360	30E/360 — day 31 becomes day 30
B_252	"B252"	252	Business/252 — business days ÷ 252
A_336	"A336"	336	Actual/336 — actual days ÷ 336

BusinessDayConventionEnum

Defines how non-business days are handled. Used in `PamContractTerms.BusinessDayConvention`.

Value	Shift Strategy	Calc Strategy	Description
NOS	Same	ShiftCalc	No adjustment
CSF	Following	CalcShift	Settle forward, calc on original
CSMF	Modified Following	CalcShift	Settle modified forward, calc on original
CSP	Preceding	CalcShift	Settle backward, calc on original
CSMP	Modified Preceding	CalcShift	Settle modified backward, calc on original
SCF	Following	ShiftCalc	Settle and calc on shifted forward date
SCMF	Modified Following	ShiftCalc	Settle and calc on modified forward date
SCP	Preceding	ShiftCalc	Settle and calc on shifted backward date
SCMP	Modified Preceding	ShiftCalc	Settle and calc on modified backward date
NO_ADJUST	Same	ShiftCalc	Legacy alias for NOS

Calendar

Defines which days are business days. Used in `PamContractTerms.Calendar`.

Value	Description
NC	No Calendar — all days are business days
MF	Monday to Friday — weekdays only
MFH	Monday to Friday with Holidays — weekdays minus holiday set

EndOfMonthConventionEnum

Defines how schedule dates align to month-ends. Used internally by the `EndOfMonthAdjuster`.

Value	Description
SD	Same Day — no adjustment
EOM	End of Month — snap to last day of month (if conditions are met)

ContractTypeEnum

All ACTUS contract types. Only PAM is implemented in this codebase.

Value	Name	Status
PAM	Principal at Maturity	**Implemented**
ANN	Annuity	Not implemented
NAM	Negative Amortization	Not implemented
LAM	Linear Amortization	Not implemented
LAX	Linear Amortization with Extension	Not implemented
CLM	Call Money	Not implemented
UMP	Undefined Maturity Profile	Not implemented
CSH	Cash	Not implemented
STK	Stock	Not implemented
COM	Commodity	Not implemented
SWAPS	Plain Vanilla Interest Rate Swap	Not implemented
SWPPV	Plain Vanilla IR Swap PV	Not implemented

Value	Name	Status
FXOUT	Foreign Exchange Outright	Not implemented
CAPFL	Caplet/Floorlet	Not implemented
FUTUR	Future	Not implemented
OPTNS	Option	Not implemented
CEG	Credit Enhancement Guarantee	Not implemented
CEC	Credit Enhancement Collateral	Not implemented
BCS	Basic Credit Support	Not implemented

ContractPerformance

Tracks the performance status of a contract over its lifetime.

Value	Name	Description
PF	Performing	Contract is operating normally
DL	Delayed	Payments are delayed
DQ	Delinquent	Payments are significantly overdue
DF	Default	Contract is in default
MA	Matured	Contract has reached maturity
TE	Terminated	Contract was terminated early

FeeBasis

Determines how fees are calculated.

Value	Name	Description
A	Absolute	Fixed amount per period (e.g., $100 per quarter)
N	Notional	Percentage of principal (e.g., 0.1% of notional per quarter)

CyclePointOfInterestPayment / CyclePointOfRateReset

Determines whether the event is at the beginning or end of its cycle period.

Value	Description
B	Beginning of period
E	End of period

Interfaces

IContractTerms

Base interface for all contract term types.

Member	Type	Description
ContractID	string	Unique contract identifier
ContractType	string	ACTUS contract type code
Currency	string	ISO currency code
StatusDate	DateTime	Observation/analysis date
MaturityDate	DateTime	Contract end date

Implemented by: `PamContractTerms`

IContractScheduler<TTerms>

Generic interface for contract processors.

Method	Returns	Description
Schedule(DateTime to, TTerms terms)	List<ContractEvent>	Generate raw events
Apply(List<ContractEvent>, TTerms, RiskFactorModel)	List<ContractEvent>	Evaluate events with state

Implemented by: `PrincipalAtMaturity` (with TTerms = PamContractTerms)

IDayCountConventionProvider

Interface for day count convention implementations.

Method	Returns	Description
DayCount(DateTime start, DateTime end)	double	Raw day count between dates
DayCountFraction(DateTime start, DateTime end)	double	Year fraction between dates

Implemented by: `ActualThreeSixtyFiveFixed`, `ActualThreeSixty`, `ActualActualISDA`, `ThirtyEThreeSixty`, `ThirtyEThreeSixtyISDA`, `BusinessTwoFiftyTwo`, `ActualThreeThirtySix`, `TwentyEightThreeThirtySix`

IBusinessDayConvention

Interface for business day shift strategies.

Method	Returns	Description
Shift(DateTime date)	DateTime	Shift date to nearest business day

Implemented by: `Same`, `Following`, `ModifiedFollowing`, `Preceeding`, `ModifiedPreceeding`

IShiftCalcConvention

Interface for calc/shift strategies.

Method	Returns	Description
Shift(DateTime time, IBusinessDayConvention bd)	DateTime	Apply calc convention

Implemented by: `ShiftCalc` (applies business day shift), `CalcShift` (returns original date)

IEndOfMonthConvention

Interface for end-of-month adjustments.

Method	Returns	Description
Shift(DateTime date)	DateTime	Apply EOM adjustment

Implemented by: `EndOfMonth`, `SameDay`

ICycleAdjusterProvider

Interface for cycle arithmetic (advancing/retreating by one cycle).

Method	Returns	Description
PlusCycle(DateTime time)	DateTime	Advance by one cycle
MinusCycle(DateTime time)	DateTime	Retreat by one cycle

Implemented by: `PeriodCycleAdjuster`, `WeekdayCycleAdjuster`

Classes

PamContractTerms (sealed)

The complete domain model for a PAM contract. Over 40 properties organized into: identity and dates, principal and interest, payment cycles, rate reset parameters, fee parameters, scaling parameters, and convention selections. Includes `FromDictionary()` factory for JSON deserialization and `RoleSign` cached property.

File: `ActusInsurance.Core/Models/PamContractTerms.cs` (~306 lines)

ContractEvent (sealed)

Represents a single event in a contract's life. Properties: Time, ScheduleTime, Type, Currency, Payoff, and state snapshot fields. Methods: `Evaluate()` (orchestrates payoff + state update + snapshot), `ComputePayoff()`, `UpdateState()`, and helper methods for each event type. Implements `IComparable<ContractEvent>` for deterministic sorting.

File: `ActusInsurance.Core/Events/ContractEvent.cs`

PrincipalAtMaturity (static)

The PAM contract processor. Two static methods: `Schedule()` generates events from terms, `Apply()` evaluates events with market data. Contains `InitializeStateSpace()` for state initialization and `CreateBusinessDayAdjuster()` factory.

File: `ActusInsurance.Core.CPU/Contracts/PAM.cs` (~346 lines)

RiskFactorModel (sealed)

Market data storage and lookup. Stores constant rates and time series. `GetRate()` uses a four-step priority: constant → exact match → LOCF → default (0.0).

File: `ActusInsurance.Core/Externals/RiskFactorModel.cs`

DayCountCalculator

Factory class that takes a convention string and produces the corresponding `IDayCountConventionProvider` implementation. Maps: "AA" → ActualActualISDA, "A360" → ActualThreeSixty, "A365" → ActualThreeSixtyFiveFixed, "30E360" → ThirtyEThreeSixty, "30E360ISDA" → ThirtyEThreeSixtyISDA, "B252" → BusinessTwoFiftyTwo, "A336" → ActualThreeThirtySix, "28336" → TwentyEightThreeThirtySix.

File: `ActusInsurance.Core/Conventions/DayCount/DayCountCalculator.cs`

BusinessDayAdjuster (sealed)

Coordinates business day and calc/shift conventions. Two methods: `ShiftEventTime()` for settlement date, `ShiftCalcTime()` for calculation date. Internally delegates to `IBusinessDayConvention` and `IShiftCalcConvention` instances.

File: `ActusInsurance.Core/Conventions/BusinessDay/BusinessDayAdjuster.cs`

EndOfMonthAdjuster (sealed)

Coordinates end-of-month adjustment. Evaluates two conditions (last-day-of-month anchor + month-based cycle) before activating EOM shift. Falls back to SameDay if conditions are not met.

File: `ActusInsurance.Core/Conventions/EndOfMonth/EndOfMonthAdjuster.cs`

ScheduleFactory (static)

Generates periodic date sequences. `CreateSchedule()` takes start, end, cycle string, EOM flag, and include-end flag. Handles null cycles, forward generation, EOM adjustment, and long/short stub logic.

File: `ActusInsurance.Core/Time/ScheduleFactory.cs`

CycleAdjuster

Dispatches between period-based and weekday-based cycle arithmetic. Wraps either a `PeriodCycleAdjuster` or a `WeekdayCycleAdjuster`.

File: `ActusInsurance.Core/Time/CycleAdjuster.cs`

Calendar Providers

Class	Calendar	Business Days
BusinessDayCalendarProvider	MF (base)	Monday–Friday
MondayToFridayCalendar	MF	Monday–Friday
MondayToFridayWithHolidaysCalendar	MFH	Monday–Friday minus holidays (HashSet)
NoHolidaysCalendar	NC	All days

Directory: `ActusInsurance.Core/Time/Calendar/`

Structs

StateSpace

Value struct carrying the complete contract state between events: NotionalPrincipal, NominalInterestRate, AccruedInterest, FeeAccrued, NotionalScalingMultiplier, InterestScalingMultiplier, StatusDate, ContractPerformance.

File: `ActusInsurance.Core/States/StateSpace.cs` (~30 lines)

Period

Data class for parsed cycle periods: Years (int), Months (int), Days (int). `GetMonths()` returns `Months + (Years × 12)`.

File: defined within `CycleUtils.cs`

Utility Classes

CycleUtils (static)

Cycle string parsing. Methods: `IsPeriod()`, `ParsePeriod()`, `ParseStub()`, `ParseWeekday()`, `ParsePosition()`.

StringUtils (static)

Convention string constants: LongStub ('0'), ShortStub ('1'), day count convention keys ("AA", "A360", "A365", etc.), calc/shift convention markers ("CS", "SC").

CommonUtils (static)

Null-check helper: `IsNull()` returns true for null objects or empty strings.

Constants (static)

Maximum contract lifetimes: MAX_LIFETIME (50 years), MAX_LIFETIME_STK (10 years), MAX_LIFETIME_UMP (10 years).

TimeAdjuster (static)

Hour rounding: `ToFullHours()` rounds a DateTime to the nearest full hour.

AttributeConversionException

Standard exception class for parsing/conversion errors during term deserialization.

Project Dependencies

ActusInsurance.Core

- Target: .NET 9.0
- External dependencies: **none** (pure C#)
- Role: shared abstractions, models, conventions, utilities

ActusInsurance.Core.CPU

- Target: .NET 9.0
- Dependencies: ActusInsurance.Core (project reference)
- Role: PAM contract processor

ActusInsurance.Tests.CPU

- Target: .NET 9.0
- Dependencies:
  - ActusInsurance.Core (project reference)
  - ActusInsurance.Core.CPU (project reference)
  - NUnit 4.2.2
  - Microsoft.NET.Test.Sdk 17.12.0
  - NUnit.Analyzers 4.4.0
  - NUnit3TestAdapter 4.6.0
  - coverlet.collector 6.0.2

- Test data: `Resources/actus-tests-pam.json` (42 test cases)
- Tolerance: 2×10^{-10} for double comparisons (10 decimal places after rounding)

Financial Conventions

Financial Conventions

Overview

Financial contracts use a set of conventions to handle the practical details of dates and time measurement. These conventions determine three things: how much interest accrues between two dates (day count conventions), when payments actually settle (business day conventions), and how schedule dates align to month boundaries (end-of-month conventions).

Getting conventions right is critical: the same contract evaluated with different conventions produces different cash flows. For large portfolios, convention differences aggregate into material amounts. The ACTUS standard specifies each convention precisely, and this engine implements all of them.

Concepts Before Details

Before diving into the implementations, three concepts need to be clear:

Year fraction. Interest is quoted as an annual rate (e.g., 5% per year), but payments happen at arbitrary intervals (monthly, quarterly, etc.). A "year fraction" converts the time between two dates into a fraction of a year. Multiply the year fraction by the annual rate and the principal to get the interest amount. Different conventions define "fraction of a year" differently — and that is the entire source of complexity.

Business day. A calendar day when financial markets are open and settlements can occur. Typically Monday through Friday, excluding public holidays. When a scheduled payment falls on a non-business day, a convention determines what happens: the payment might move to the next business day, the previous one, or stay put.

Calc vs. Shift. There are two separate questions about a payment that falls on a non-business day: (1) when does the payment actually settle? and (2) what dates do you use to calculate interest? These can have different answers. "CalcShift" means: calculate interest using the original date, but shift the settlement. "ShiftCalc" means: both the calculation and the settlement use the shifted date.

Day Count Conventions

A day count convention defines how to measure the time between two dates for interest calculation. The engine implements seven conventions through the `IDayCountConventionProvider` interface, which has two methods:

- `DayCount(startDate, endDate)` — the raw number of days (or adjusted days)
- `DayCountFraction(startDate, endDate)` — the year fraction (days divided by the year basis)

The `DayCountCalculator` class acts as a factory, taking a convention string and returning the correct implementation.

A/365 (Actual/365 Fixed)

The simplest convention. Count the actual calendar days between the two dates and divide by 365. Always 365 — even in leap years.

```
YearFraction = ActualDays / 365
```

Used for UK government bonds and many non-USD instruments. Example: from January 15 to April 15 (90 days) → 90/365 = 0.24658.

A/360 (Actual/360)

Count actual calendar days, divide by 360. Because the denominator is smaller than the actual year, this convention produces slightly higher interest amounts than A/365 for the same period.

```
YearFraction = ActualDays / 360
```

Used for USD money markets and LIBOR-based instruments. Example: from January 15 to April 15 (90 days) → 90/360 = 0.25000.

A/A ISDA (Actual/Actual ISDA)

The most accurate convention — it accounts for leap years. When a period falls entirely within one calendar year, divide by the actual number of days in that year (365 or 366). When a period spans multiple years, split it at the year boundary and compute each segment separately.

```
If same year:
    YearFraction = ActualDays / DaysInYear(year)

If spanning years:
    YearFraction = DaysInFirstYear / BasisOfFirstYear
                 + DaysInLastYear / BasisOfLastYear
                 + CompleteYearsInBetween
```

Where `BasisOfYear` is 366 for leap years and 365 for non-leap years.

Used for government bonds where leap-year accuracy matters. Example: from December 1, 2023 to March 1, 2024 (91 days, spanning a leap year) → 31/365 + 60/366 = 0.24867.

30E/360 (European 30/360)

An artificial convention that assumes every month has 30 days and every year has 360 days. Day 31 of any month is treated as day 30.

```
DayCount = 360 × (Y2 − Y1) + 30 × (M2 − M1) + (D2_adj − D1_adj)
YearFraction = DayCount / 360
```

Where D1_adj and D2_adj replace day 31 with day 30.

Used for many European bonds and standardized calculations. Example: from January 31 to March 31 → 360×0 + 30×2 + (30−30) = 60 days → 60/360 = 0.16667.

30E/360 ISDA

A variant of 30E/360 with an ISDA-specific rule: if either date is the last day of its month, treat it as day 30, except that the maturity date in February is not adjusted. This prevents a February 28 maturity from being treated as February 30.

The implementation requires knowledge of the maturity date, which is set via a `SetMaturityDate()` method before calculation.

B/252 (Business/252)

Instead of counting calendar days, count only business days (as determined by the calendar provider) and divide by 252 — the assumed number of business days per year.

```
YearFraction = BusinessDays / 252
```

Used in the Brazilian market. This convention requires a calendar that defines which days are business days, making it the only day count convention with an external dependency.

A/336 (Actual/336)

Count actual calendar days, divide by 336 (28 days × 12 months). This convention is specific to certain insurance applications where a "month" is always 28 days.

```
YearFraction = ActualDays / 336
```

28/336

A companion to A/336 that treats day counts similarly to 30E/360 but with a 28-day month and 336-day year. Days 28 through 31 of any month are treated as day 28, and the formula uses 336 × ΔY + 28 × ΔM + ΔD.

Business Day Conventions

When a scheduled event falls on a non-business day, the business day convention determines two things: where the event actually settles (the "shifted" date), and what date is used for interest calculations.

The engine implements this through two cooperating interfaces:

- `IBusinessDayConvention` — the shift strategy: how to move a non-business day to a business day
- `IShiftCalcConvention` — the calc strategy: whether calculations follow the shift or stay on the original date

The `BusinessDayAdjuster` class coordinates both, exposing two methods:

- `ShiftEventTime(date)` — returns the shifted settlement date
- `ShiftCalcTime(date)` — returns the date to use for interest calculations

Shift Strategies

Same (No Adjustment): The event stays on its scheduled date regardless of whether it is a business day. Used with the NOS convention.

Following: Move forward to the next business day. If an event falls on Saturday, it settles on Monday.

```
while (date is not a business day):
    date = date + 1 day
```

Modified Following: Move forward to the next business day, but if that would cross a month boundary, move backward instead. This prevents a month-end payment from settling in the next month.

```
shiftedDate = apply Following(date)
if shiftedDate.Month ≠ date.Month:
    shiftedDate = apply Preceding(date)  // go backward instead
```

Preceding: Move backward to the previous business day.

```
while (date is not a business day):
    date = date − 1 day
```

Modified Preceding: Move backward, but if that would cross a month boundary, move forward instead.

```
shiftedDate = apply Preceding(date)
if shiftedDate.Month ≠ date.Month:
    shiftedDate = apply Following(date)  // go forward instead
```

Calc/Shift Strategies

ShiftCalc: Both the event settlement and the calculation use the shifted date. The `ShiftCalcTime()` method applies the business day shift.

CalcShift: The event settles on the shifted date, but calculations use the original unshifted date. The `ShiftCalcTime()` method returns the original date unchanged.

The Nine Conventions

Combining the shift and calc strategies yields nine conventions:

Code	Shift Strategy	Calc Strategy	Description
NOS	Same	ShiftCalc	No adjustment — everything uses the original date

Code	Shift Strategy	Calc Strategy	Description
CSF	Following	CalcShift	Settle forward, calculate on original
CSMF	Modified Following	CalcShift	Settle modified forward, calculate on original
CSP	Preceding	CalcShift	Settle backward, calculate on original
CSMP	Modified Preceding	CalcShift	Settle modified backward, calculate on original
SCF	Following	ShiftCalc	Both settle and calculate on the shifted forward date
SCMF	Modified Following	ShiftCalc	Both on the modified forward date
SCP	Preceding	ShiftCalc	Both on the shifted backward date
SCMP	Modified Preceding	ShiftCalc	Both on the modified backward date

The "CS" prefix (Calc-Shift) means calculations use the original date. The "SC" prefix (Shift-Calc) means calculations follow the shift. The suffix indicates the direction: "F" for following, "MF" for modified following, "P" for preceding, "MP" for modified preceding.

When the Convention Matters

Consider a quarterly interest payment scheduled for Saturday, March 31. With convention SCF (ShiftCalc Following), the payment settles on Monday, April 2, and the interest accrual period runs to April 2. With CSF (CalcShift Following), the payment also settles on April 2, but the interest accrual period runs to March 31 (the original date). The difference in interest can be non-trivial.

End-of-Month Convention

When a schedule cycle starts on the last day of a month (e.g., January 31 or February 28), the end-of-month convention determines whether subsequent dates also snap to month-ends.

The `EndOfMonthAdjuster` evaluates two conditions before activating:

1. The reference date (cycle anchor) must be the last day of its month
2. The cycle must have a month-based period (months > 0)

If both conditions are met and the EOM convention is enabled, every generated schedule date is shifted to the last day of its month. Otherwise, the SameDay convention applies (no adjustment).

EOM Convention	Starting Jan 31	February	March	April
EOM (enabled)	Jan 31	Feb 28/29	Mar 31	Apr 30
SD (disabled)	Jan 31	Feb 28/29	Mar 31	Apr 30*

*In practice, SD keeps the day-of-month where possible (e.g., 31st → 28/29 → 31st → 30th).

Implementation Detail

The `EndOfMonth` shift implementation:

```
Shift(date) → new DateTime(date.Year, date.Month, DaysInMonth(date.Year,
date.Month))
```

The `SameDay` implementation returns the date unchanged.

Calendars

The calendar determines which days are business days. Three calendar types are supported:

NC — No Calendar

All days are business days. No shifting ever occurs. This is the default and is used when no business day adjustments are needed.

MF — Monday to Friday

Standard weekday calendar. Saturdays and Sundays are non-business days. No holiday support.

MFH — Monday to Friday with Holidays

Weekdays minus a specific set of holiday dates. The implementation stores holidays as a `HashSet<DateTime>` for O(1) lookup. A day is a business day if it is Monday–Friday and not in the holiday set.

Calendar Providers

All calendars extend the `BusinessDayCalendarProvider` base class, which defines the `IsBusinessDay(DateTime)` method. The base implementation returns true for weekdays (used by MF). The `NoHolidaysCalendar` always returns true (used by NC). The `MondayToFridayWithHolidaysCalendar` additionally checks against the holiday set.

How Conventions Flow Through the Engine

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    TERMS["Contract Terms<br/>(convention selections)"] --> BDA["BusinessDayAdju
    TERMS --> DCC["DayCountCalculator<br/>(year fraction computation)"]
    TERMS --> EOM["EndOfMonthAdjuster<br/>(schedule date snapping)"]

    BDA --> SCHED["Schedule Generation<br/>(shift event dates)"]
    EOM --> SCHED

    BDA --> EVAL["Event Evaluation<br/>(ShiftCalcTime for accrual dates)"]
    DCC --> EVAL

    SCHED --> EVENTS["Events with two times:<br/>Time (shifted) + ScheduleTime (
    EVENTS --> EVAL
```

The convention selections in the contract terms instantiate the appropriate implementations at the start of processing. These implementations are then threaded through schedule generation (for date shifting and EOM snapping) and event evaluation (for year fraction calculation using the correct date pair).

Continue Reading

- Schedule Generation — how cycle strings produce periodic date sequences
- State Machine — how conventions affect interest accrual between events
- PAM Contract — where conventions are used in the full contract pipeline
- Technical Reference — enum values for all convention types

CLI Demo Tool — Monte Carlo Portfolio Valuation

CLI Demo Tool — Monte Carlo Portfolio Valuation

Overview

The PamMonteCarlo50Y CLI tool demonstrates the complete pipeline: portfolio generation (or loading), scenario generation, GPU (or CPU) evaluation, sink aggregation, and reporting. It is the primary way to exercise the system end to end.

Modes of Operation

The CLI supports two input modes:

Synthetic mode — generates a random portfolio and Monte Carlo scenarios from parameters:

```
PamMonteCarlo50Y --backend gpu --contracts 10000 --scenarios 1000 --
months 600 --seed 42
```

Input-directory mode — loads a pre-built portfolio and scenarios from CSV/JSON files:

```
PamMonteCarlo50Y --backend gpu --input ./samples/input --out ./results
```

Key CLI Options

Option	Description	Default
`--backend`	cpu, gpu, or both	both
`--contracts`	Number of synthetic contracts	1000
`--scenarios`	Number of Monte Carlo scenarios	100
`--months`	Projection horizon in months	600 (50 years)

Option	Description	Default
`--seed`	Random seed for reproducibility	42
`--calcDateIndex`	Boundary between historical and forward rates	0
`--runs`	Path to runs.json for multi-run configuration	—
`--input`	Input directory for pre-built portfolios	—
`--out`	Output directory	./out
`--reporting`	Enable detailed reporting exports	false
`--export-fact`	Export long-format fact table	false
`--export-portfolio`	Export generated portfolio to CSV	false
`--metadata`	Path to contract metadata CSV for joined reporting	—

Execution Pipeline

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    INPUT["Input<br/>(synthetic or directory)"] --> PORTFOLIO["Portfolio<br/>Par
    INPUT --> SCENARIOS["Scenarios<br/>Rate paths via Vasicek model"]
    PORTFOLIO --> ORCHESTRATE["DemoOrchestrator<br/>Routes to CPU or GPU engine"
    SCENARIOS --> ORCHESTRATE
    ORCHESTRATE --> EVAL["Evaluation<br/>PayoffCube (GPU) or CPU loop"]
    EVAL --> SINKS["Sink Aggregation<br/>Per-contract statistics"]
    SINKS --> EXPORT["Export<br/>CSV + JSON reports"]
```

Portfolio Generator

The synthetic portfolio generator creates realistic PAM contracts with varied parameters:

- Notional principal (randomised within configurable range)
- Interest rates (randomised, possibly floating with rate resets)
- Maturity dates (varied horizons)
- Payment frequencies (monthly, quarterly, semi-annual, annual)
- Day count conventions (varied across contracts)

The generator is **deterministic** — the same seed always produces the same portfolio.

Vasicek Rate Generator

Monte Carlo scenarios are generated using the Vasicek interest rate model, which produces mean-reverting rate paths characterised by:

Parameter	Description
κ (kappa)	Mean reversion speed
θ (theta)	Long-term mean rate
σ (sigma)	Volatility
r_0	Initial rate

Each scenario is an independent rate path. The generator uses a seeded PRNG for full reproducibility.

Reporting Exports

When `--reporting true` is enabled, the tool produces:

File	Content
`{runId}_portfolio_by_scenario.csv`	RunId, ScenarioId, PortfolioPV
`{runId}_contract_summary.csv`	Per-contract statistics: Mean, StdDev, P05, P25, Median, P75, P95, ES95, ES99
`{runId}_fact_results_long.csv`	Long-format fact table (when --export-fact true)
`runs.csv`	Run dimension table with metadata

These outputs are designed to be opened directly in Excel or imported into BI tools for further analysis.

Input Directory Structure

When using `--input`, the directory must follow this structure:

```
input_dir/
├── portfolio.csv                    — Contract terms in CSV format
├── scenarios/
```

```
│   ├── scenario_set.json           — Scenario metadata
│   └── riskfactors/
│       ├── interest_rate_prior.csv  — Historical rates (before
calcDateIndex)
│       └── interest_rate_after.csv  — Forward rates (scenario-specific)
├── runs.json                       — (optional) Multi-run definitions
└── contract_metadata.csv           — (optional) Extra contract
attributes for reporting
```